

The SILK SKIES Blueprint

A guided tour of how the game is built: the floors of the building, every object on stage, how they talk, and how to change things without breaking them. Written for the person who directs the work.

~5,000 lines of game code 387 automated tests 34 data files 20 diagrams

3 milestones built, 1 to go for the loop

CONTENTS

1	The five-minute tour	11	How parts talk
2	The building map	12	Screens and tools
3	The cast of objects	13	Three guided walks
4	Data: the recipe book	14	How to extend
5	Stats and modifiers	15	Testing and tooling
6	Sailing	16	Conventions and gotchas
7	Combat	17	Health check
8	Inventory and refit	18	Spec status
9	Enemy AI and bosses	19	Glossary
10	Game flow and runs	A	File index

How to use this report

It is long on purpose. You do not read it front to back; you pick a route.

IF YOU WANT TO...	READ	TIME
Hold a conversation about the code	Section 1	5 min
Know where to look when something is wrong	Sections 2 and 3	15 min
Direct new work and judge its size	Sections 2, 14 and 17	20 min
Understand how a feature really works	The matching deep dive (4 to 12), then the walks in 13	10 min each
Check the project against the spec	Section 18	5 min
Look up a word or a file	Sections 19 and A	as needed

Diagrams are numbered in the captions (FIG). Their colours always mean the same layer of the building; the key is in section 2.

The five-minute tour

Read this section and you can hold a conversation about the code. Everything after it is a deeper look at one room of the building.

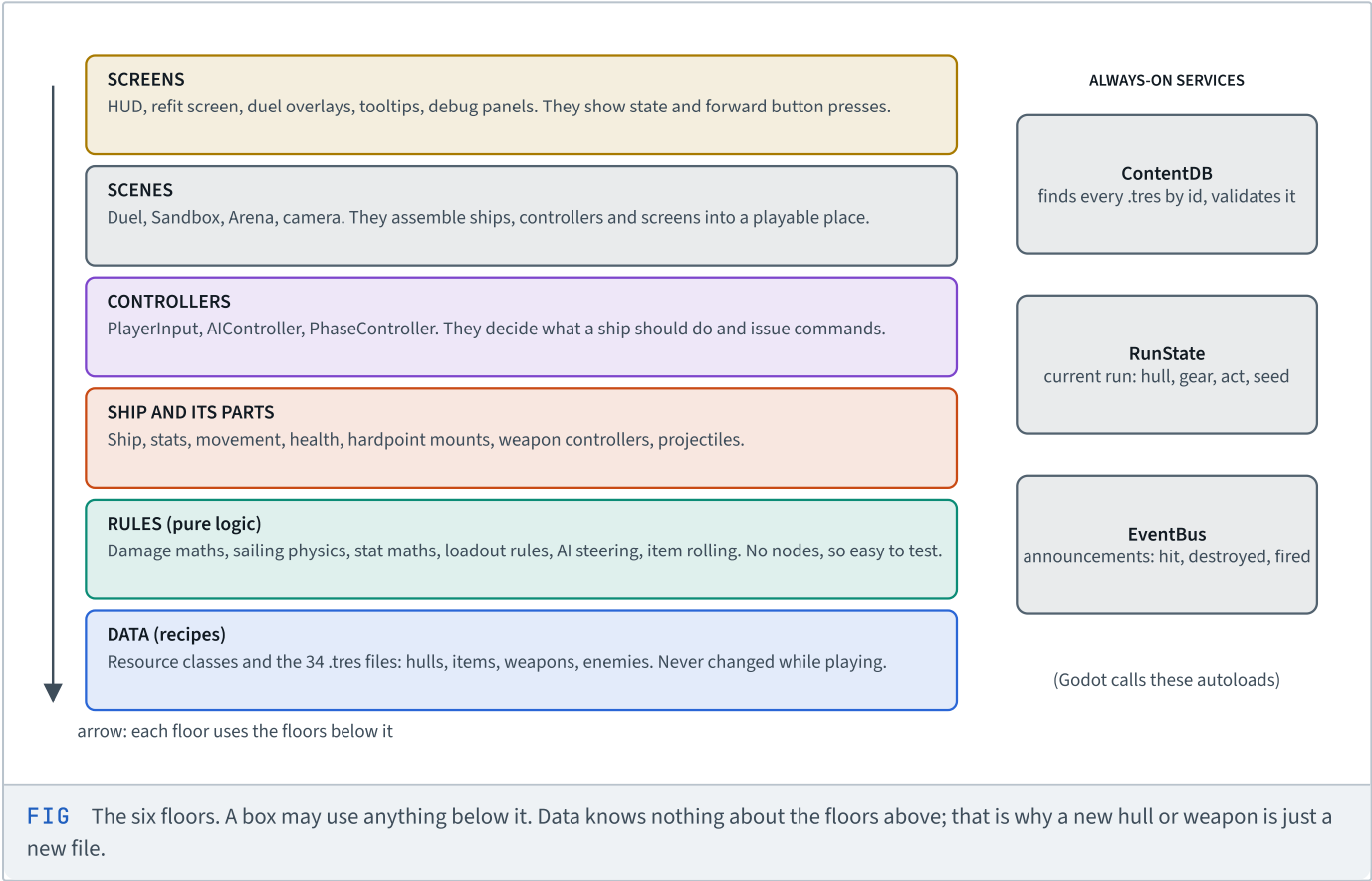
What exists right now

SILK SKIES is a 3D top-down duel game: you sail a spaceship around a circular arena, fire weapons at an enemy ship, and refit your gear between fights. Three milestones are done. **M1** gave us sailing and the data system. **M2** added combat. **M2.5** added the inventory and refit screen. **M3** added enemy AI, a boss, and a three-duel gauntlet. **M4**, the reward-and-loop milestone, is next.

Everything on screen is still placeholder boxes. That is on purpose: the code underneath is real, tested, and built so that real art can be swapped in later without touching the rules.

The building, in one picture

Think of the project as a building with six floors. The ground floor holds *recipes* (data). Each floor above uses the floors below it and never reaches back down to change them. On the side, three *always-on services* (autoloads) are reachable from every floor.



Seven ideas that explain most of the code

#	IDEA	WHAT IT MEANS IN PRACTICE
1	Recipes vs. dishes	A <code>.tres</code> file (a recipe) is authored and never changes. A specific drop of an item is a separate <i>instance</i> (a dish) with its own rarity and rolled stats. See section 4.
2	One Ship for everybody	The player and every enemy are the same <code>Ship</code> scene. Only the controller attached to it differs. Enemies are not special code.

#	IDEA	WHAT IT MEANS IN PRACTICE
3	Controllers only give orders	Keyboard, AI and debug bots all use the same five commands: throttle, turn, hard burn, aim, trigger. The ship decides how to obey.
4	Rules live in pure functions	Damage, sailing, stats and loadout rules are plain classes with no scene nodes. That is why 387 tests run in 15 seconds.
5	Each part has one job	Stats compute numbers, Movement moves, Health tracks HP, a WeaponController runs one weapon. If you are hunting a bug, the job tells you the file.
6	Announce, don't command	When something happens (a hit, a death) the code emits a signal. Screens and effects listen. Gameplay never reaches into the UI.
7	Tests guard the walls	Every rule has tests. A fight can even be simulated headlessly at 340x speed to measure balance.

WHY THIS MATTERS TO YOU

Almost every feature you will ask for next is either *a new data file* (new hull, weapon, enemy) or *a new rule in one place*. The map in the next section tells you which, and where.

The building map

Where every file lives, how big the rooms are, and a lookup table for the question you will ask most:
"something is wrong with X, where do I look?"

Folder layout

```
silk-skies/
├─ project.godot           engine settings: autoloader, input keys, physics layers
├─ 2026-10-05-ship-roguelite-design.md   the design spec (the client's contract)
├─ scripts/               ALL game code (~5,000 lines)
│   ├── data/             22 files resource CLASSES: HullData, ItemData, EnemyData, StatMod ...
│   ├── systems/          13 files pure rules: DamageModel, ShipLoadout, AISteering, EnemyFactory ...
│   ├── autoload/         3 files always-on services: ContentDB, RunState, EventBus
│   ├── ship/             15 files what a ship IS: Ship, Movement, Health, Weapon, Projectile, AI ...
│   ├── arena/            1 file  the circular arena + boundary
│   ├── duel/             1 file  the 1v1 duel scene logic
│   ├── ui/               10 files HUD, refit screen, duel overlays, tooltips, damage numbers
│   └─ debug/             4 files  the sandbox and its panels (F1 / F2)
├─ scenes/               .tscn scene files (assembled objects)
│   ├── ship/ship.tscn    THE ship; ship/models/ = 7 placeholder hull models
│   ├── arena/           arena + asteroid
│   ├── duel/duel.tscn   a duel
│   └─ debug/sandbox.tscn the sandbox (currently the main scene)
├─ data/                 34 .tres RECIPE FILES: hulls, weapons, projectiles, items,
│                       enemies, AI profiles, roll pools, act
├─ shaders/grid.gdshader the floor grid
├─ tests/                ~4,000 lines of tests
│   ├── unit/            pure-logic tests (no scenes)
│   ├── integration/     tests with real ships and real physics
│   ├── helpers/ship_test.gd shared test toolkit
│   └─ probe/            the balance probe (not part of normal runs)
└─ addons/gut/           the test framework (third-party, vendored)
```

Two folders are easy to confuse. `scripts/data/` holds the *classes* that describe what a hull or weapon *is* (code). `data/` holds the actual *files* (content). Adding a weapon means touching `data/` only.

Colour key used in every diagram

- Screens
- Scenes
- Controllers
- Ship and parts
- Rules
- Data
- AI

Orange arrows always mean something *moves*: a command, a piece of data, or an event. Grey arrows mean "uses" or "owns".

Where do I look when...

SYMPTOM	START HERE	WHY
Ship feels wrong to steer (slidey, too slow, turns badly)	<code>scripts/ship/sailing_model.gd</code> and the hull's <code>.tres</code>	All sailing physics is in one pure class; numbers come from the hull file or the F1 tuning panel.
A weapon does too much / too little damage	<code>data/weapons/*.tres</code> and <code>data/projectiles/*.tres</code> ; for the type table, <code>scripts/systems/damage_model.gd</code>	Numbers are data. The shield/armor/hull multipliers are constants at the top of the damage model.
A weapon fires at the wrong moment, or won't fire	<code>scripts/ship/weapon_controller.gd</code>	It owns cooldown, arc, charge, lock-on and salvo timing.

SYMPTOM	START HERE	WHY
Shots pass through things / hit friends	<code>scripts/ship/projectile.gd</code>	Collision is a ray-cast along each step; teams are checked there.
A drag-and-drop does the wrong thing	<code>scripts/systems/ship_loadout.gd</code> (rules) then <code>scripts/ui/item_slot.gd</code> (UI)	The rules decide what is legal; the slot only asks and forwards.
Stats on the refit screen don't match the ship	<code>scripts/systems/loadout_stats.gd</code> , <code>stat_aggregator.gd</code> , <code>ship.gd</code> (<code>_apply_loadout</code>)	That chain turns equipped items into final numbers.
Enemy behaves stupidly (stands still, won't shoot)	<code>scripts/ship/ai_controller.gd</code> and <code>data/ai/*.tres</code>	Behaviour is code; <i>style</i> is the profile numbers.
Boss phase didn't trigger	<code>scripts/ship/phase_controller.gd</code> and the enemy's <code>.tres</code>	Thresholds and added items live in the enemy file.
Duel flow is broken (buttons, pause, results)	<code>scripts/duel/duel.gd</code> and <code>scripts/ui/duel_ui.gd</code>	Duel owns the state machine; DuelUI only reports button presses.
Game won't start / content errors in the console	<code>scripts/autoload/content_db.gd</code> , <code>scripts/systems/content_validator.gd</code>	Every data file is checked at startup and problems are printed.
Something is invisible, mis-coloured, overlapping	<code>scripts/ship/hardpoint_mount.gd</code> , <code>scripts/ui/*</code> , <code>scenes/ship/models/*.tscn</code>	Visuals are placeholders built in code and in small scene files.
A test fails after my change	<code>tests/</code> mirrors <code>scripts/</code> ; the file name tells you the unit	e.g. <code>test_damage_model.gd</code> guards <code>damage_model.gd</code> .

How big is each floor?

FLOOR	FILES	LINES	LARGEST ROOMS
Data classes	22	~430	<code>hull_data.gd</code> (61)
Rules (systems)	13	~770	<code>ship_loadout.gd</code> (175), <code>content_validator.gd</code> (164)
Autoloads	3	~160	<code>content_db.gd</code> (78)
Ship and parts	15	~1,570	<code>weapon_controller.gd</code> (348), <code>ship.gd</code> (264), <code>projectile.gd</code> (161)
Screens (UI)	10	~1,130	<code>duel_ui.gd</code> (241), <code>inventory_screen.gd</code> (209)
Scenes' logic	2	~240	<code>duel.gd</code> (187), <code>arena.gd</code> (49)
Debug / sandbox	4	~650	<code>sandbox.gd</code> (350)
Tests	26	~3,975	<code>test_weapon_controller.gd</code> (451)

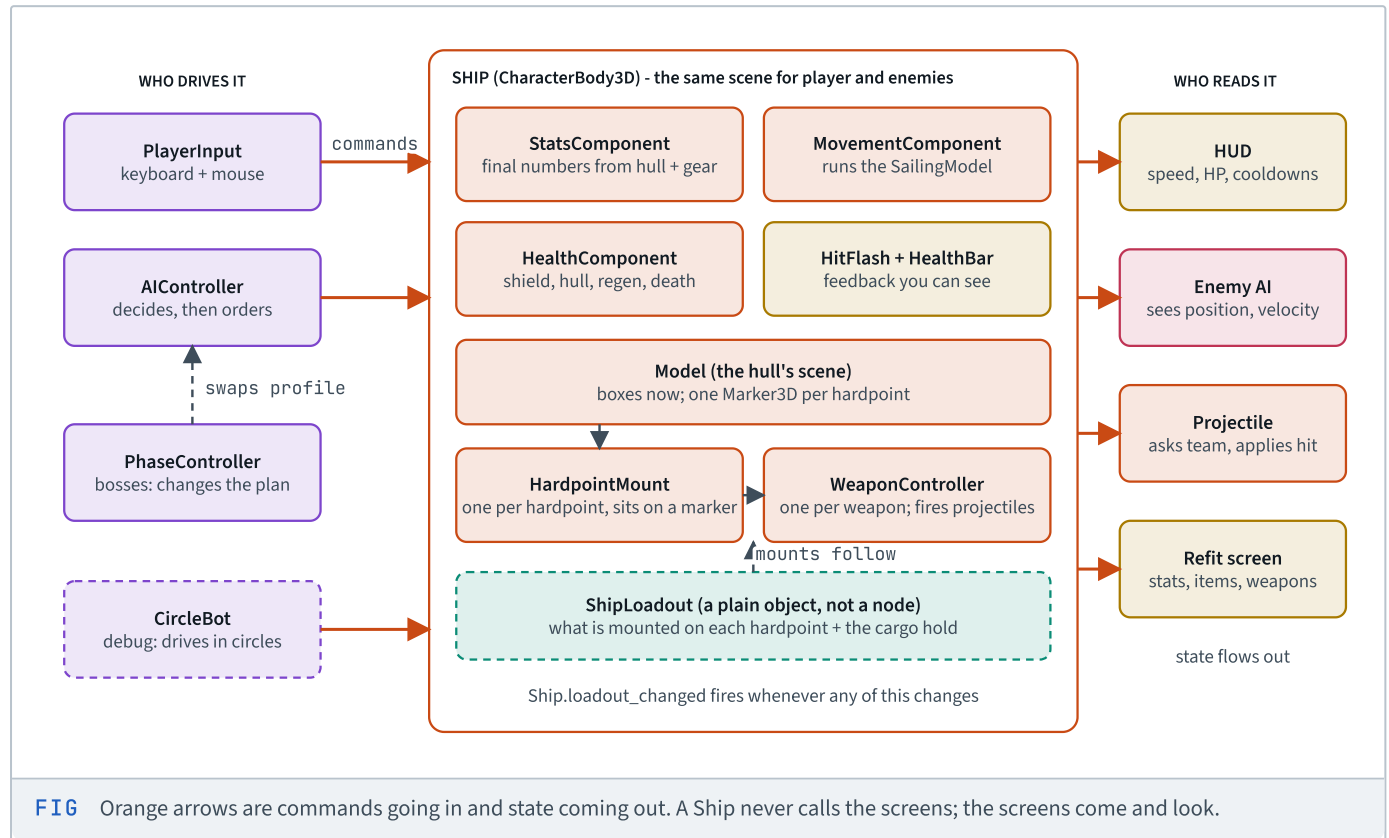
READING THE SIZES

The code is small and flat. The three biggest files (`weapon_controller` , `sandbox` , `ship`) are also the three busiest crossroads. If a file ever feels too big to hold in your head, those are the candidates to split; section 17 lists them.

The cast: every object in the game

Here is who is on stage, what each one does, and who it talks to. Start with the picture of a single Ship, because almost everything hangs off it.

Anatomy of a Ship



A Ship is built like this: the scene file `scenes/ship/ship.tscn` provides the fixed parts (Model, Stats, Movement, Health, HitFlash, HealthBar). `Ship.setup(hull)` then fills in the rest from data: it loads the hull's model, builds the collision shape, creates one mount per hardpoint, and computes stats. A controller is added afterwards by whoever spawns the ship (the Duel adds `PlayerInput` to the player; `EnemyFactory` adds `AIController` to enemies).

COMMAND API: THE FIVE ORDERS

```
set_throttle(-1..1) · set_turn(-1..1) · request_hard_burn(direction) · aim_at(point) ·  
set_trigger(mode, held)
```

Nothing else is needed to fly and fight. Add a future fighter-launch or ability and it becomes a sixth order, in one place.

The ship's parts, one card each

Ship

scripts/ship/ship.gd (264 lines)

The hub. Owns the command API, the loadout, the team, and ties the parts together.

Owns	<code>loadout</code> (what is mounted), <code>team</code> (player/enemy), <code>mounts[]</code> , trigger state, aim point
Talks to	<code>StatsComponent</code> , <code>MovementComponent</code> , <code>HealthComponent</code> , every <code>HardpointMount</code>
Emits	<code>hull_changed</code> , <code>loadout_changed</code> , <code>burn_used</code> , <code>died</code>
Key moves	<code>setup()</code> builds from a hull; <code>_apply_loadout()</code> re-syncs weapons and stats after any gear change; <code>enemies()</code> finds living ships on the other team
Does not	decide what to do (controllers), run physics (Movement), or compute damage (Health)

StatsComponent

scripts/ship/stats_component.gd

Holds the ship's final numbers: hull's base values plus every modifier from gear.

Input	the hull's base stats and a list of <code>StatMod</code> (modifiers)
Output	<code>values</code> dictionary: <code>thrust</code> , <code>max_speed</code> , <code>max_shield</code> , <code>armor</code> ...
Signal	<code>stats_changed</code> so Health can react to a bigger shield
Never	edits the <code>HullData</code> ; the tuning panel changes a copy

MovementComponent + SailingModel

movement_component.gd, sailing_model.gd (114)

Turns throttle and turn commands into motion. The maths is in SailingModel; the component just applies it.

<code>SailingModel</code>	pure class: thrust, drag, speed-scaled turning, sideways grip, Hard Burn. No nodes, fully tested
<code>MovementComponent</code>	feeds the model each physics tick, then calls <code>move_and_slide()</code> so asteroids and other ships collide
Reads	final stats from <code>StatsComponent</code>

HealthComponent

scripts/ship/health_component.gd

Shield and hull points. Receives hits, regenerates shield, announces death.

Asks	<code>DamageModel.resolve()</code> for what a hit does to each layer
Emits	<code>damaged(result)</code> , <code>destroyed(source)</code> ; also EventBus <code>ship_hit</code> / <code>ship_destroyed</code>
Quirk	when max shield grows (new capacitor) the current shield grows with it; when it shrinks it is clamped
Sandbox	has a <code>god_mode</code> flag: hits are reported but nothing is lost

HardpointMount

scripts/ship/hardpoint_mount.gd

The physical end of a hardpoint: a little cube when empty, a swinging barrel when a weapon is fitted.

Owns	the barrel visual, the charge glow, the optional firing-arc wedge
Creates	its <code>WeaponController</code> when a weapon is equipped
Note	currently mixes visuals with a little logic; real art will replace the visual half

WeaponController

scripts/ship/weapon_controller.gd (348, the biggest file)

Runs one weapon: cooldown, firing arc, auto-aim, manual charge, lock-on, salvos. Spawns projectiles.

Three modes	Auto picks targets itself, Manual follows the cursor and the left mouse button (the Railgun charges), Lock-on locks while right mouse is held and fires on release
Reads	the ship's trigger flags and aim point; the weapon's stats (damage, fire rate, range, speed)
Emits	<code>fired</code> ; EventBus <code>weapon_fired</code>
Also draws	the lock-on ring marker (placeholder)

Projectile

scripts/ship/projectile.gd (161)

A bolt, slug or missile. Moves every tick and ray-casts the distance it just travelled, so fast shots can't skip through ships.

Rules	skips its own team, stops on asteroids, can pierce N ships, can home on a target, can explode (blast radius)
Hits	<code>ship.health.apply_hit(...)</code> , nothing else
Lives	under a node in group <code>projectile_root</code> , never under the ship

Controllers: the ones giving orders

PlayerInput

scripts/ship/player_input.gd (31)

Translates your keys and mouse into the five orders.

Keys	W/S throttle, A/D steer, Shift hard burn (held direction keys choose which way), mouse aim, left button = manual weapons, right button = lock-on
Detail	ignores the mouse while it hovers over a UI panel, so clicking a slider doesn't fire

AIController

scripts/ship/ai_controller.gd (155)

The enemy's brain. Same orders as the player, driven by an AIProfile.

Loop	pick target, see it slightly late, steer toward a point on a circle around it, pull triggers when a weapon can shoot, dodge incoming shots
Styles	come from <code>data/ai/*.tres</code> : brawler, sniper, warden guard, warden enraged
Details	section 9

PhaseController

scripts/ship/phase_controller.gd (51)

Boss-only. Watches hull and switches the plan when thresholds pass.

On trigger	swaps the AI profile, mounts extra weapons, refills the shield, plays a ring, announces on EventBus
Guards	ignores the killing blow so a one-shot kill never starts a phase

The world and the feedback

Arena (+ Asteroid)

scripts/arena/arena.gd, scenes/arena/*.tscn

The circular play space: grid floor, red edge ring, five asteroids for cover.

Boundary	each physics tick it pulls any ship past the edge back and cancels the outward speed, so ships slide along the rim
Asteroids	static bodies on the <code>world</code> physics layer; they block projectiles and ships

FollowCamera

scripts/ship/follow_camera.gd

Steep top-down camera that follows the player with a small look-ahead.

Fixed	north-up, never rotates with the ship
Tweakable	height (mouse wheel in the sandbox), pitch, follow speed

Feedback pieces

health_bar.gd, hit_flash.gd, damage_numbers.gd, vfx.gd

Everything that makes a hit readable without gameplay knowing it exists.

HealthBar	two billboard bars floating above each ship (blue shield, red hull)
HitFlash	a translucent bubble that flashes blue (shield hit) or red (hull hit)
DamageNumbers	floating numbers coloured by layer: blue shield, grey armor soak, red hull; listens to EventBus
Vfx	expanding rings for explosions, deaths and boss phases

Physics layers

project.godot

Three named layers keep collisions simple.

1 ships	every Ship body
2 world	asteroids
3 projectiles	named but unused: projectiles ray-cast instead of using bodies

Data: the recipe book

All game content is written as small files that describe things, not code that does things. This is the single biggest reason adding content is cheap.

What a recipe looks like

A hull, weapon or enemy is a Godot *Resource*: a typed bundle of fields saved as a `.tres` text file. Open `data/weapons/railgun.tres` and you see roughly this (trimmed):

```
[resource]
script = ExtResource("1_weapon")      # which class describes this file (WeaponData)
id = &"railgun"                       # unique name every other file uses to refer to it
display_name = "Railgun"
size = 1                             # 0 = Small, 1 = Large turret needed
targeting = 1                         # 0 Auto, 1 Manual, 2 Lock-on
projectile = ExtResource("2_proj")    # points at data/projectiles/railgun_slug.tres
cooldown = 2.5
charge_time = 1.0                    # hold the trigger this long before it fires
range = 70.0
roll_pool = ExtResource("3_pool")     # which stat lines a drop of this weapon can roll
```

The class (`scripts/data/weapon_data.gd`) declares which fields exist and their types; the Godot editor shows them as a form. You can edit a `.tres` by hand or in the editor's inspector. Nothing in the file does anything by itself.

The family tree of data

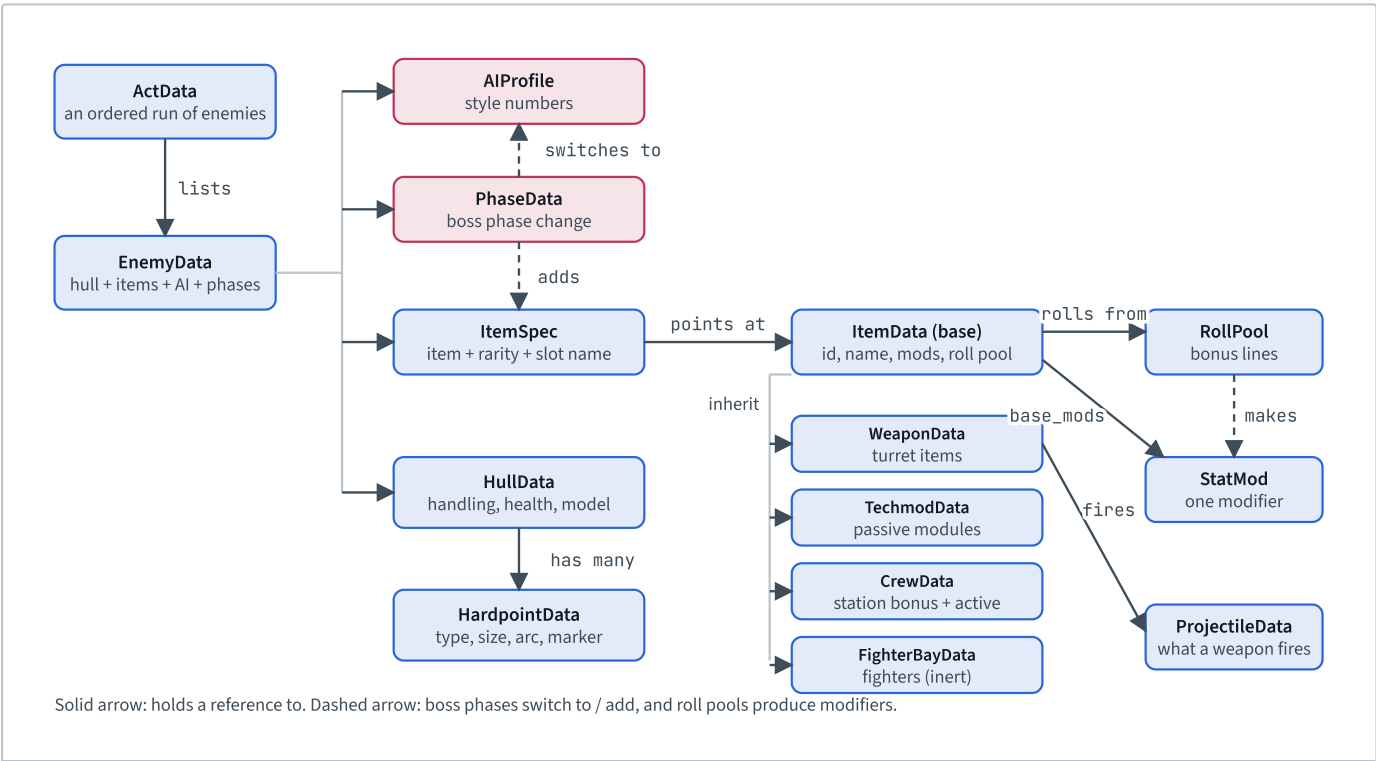
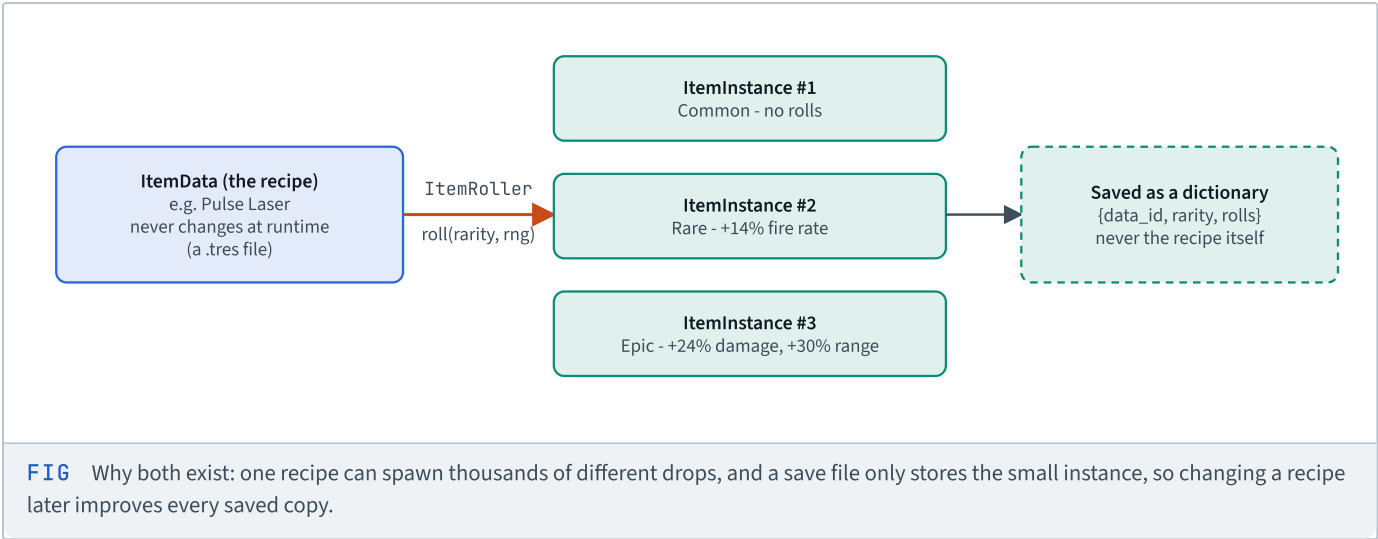


FIG Read from left to right: an Act lists Enemies; an Enemy wants a hull, a few items (by slot name) and an AI style. Items are one family sharing the same base class.

Recipe vs. dish: ItemData and ItemInstance



ItemInstance (`scripts/systems/item_instance.gd`) holds `data_id` , `rarity` and `rolls` (a list of `StatMod`). It looks the recipe up in ContentDB by id. If a recipe was deleted later, loading a save skips that item with a warning instead of crashing.

What content exists today

Hulls

ID	ROLE	HULL	SHIELD	ARMOR	TOP SPEED	TURN	HARDPOINTS
starter_frigate	Player start	300	100	3	18	120	Small 360° turret, Large 120° turret, bay, techmod, crew
debug_skiff	Light, nimble (sandbox)	150	60	1	24	170	Small turret, techmod
debug_barge	Heavy (sandbox)	500	150	5	12	70	2 Large broadside turrets (90° port / starboard), Small turret, bay, crew
raider	Enemy brawler	450	150	3	20	130	Small 360°, Large 120°
longshot	Enemy sniper	300	160	1	17	115	Large 120°, Small 360°
warden	Boss	900	250	4	12	80	2 Large 120°, 2 Small 360°
debug_dummy	Target dummy	200	150	4	-	-	none

Weapons and what they fire

WEAPON	SLOT	MODE	DAMAGE TYPE	DAMAGE	RATE / TIMING	RANGE	SPECIAL
Pulse Laser	Small	Auto	Energy	3	every 0.18 s	38	rapid short bolts
Autocannon	Small	Auto	Kinetic	4	every 0.22 s	42	3.5° spread
Railgun	Large	Manual	Kinetic	60	1.0 s charge, 2.5 s cooldown	70	pierces 2 ships; glow while charging
Missile Pod	Large	Lock-on	Explosive	14 x 4	0.8 s lock, 6 s cooldown	60	homing 140°/s, 3 m blast

Items other than weapons

ITEM	KIND	EFFECT TODAY	ACTIVE (SHELL)
Shield Capacitor	Techmod	+50 shield, +4 shield regen	-
Armor Plating	Techmod	+2 armor, +50 hull, +20 mass (heavier = slower)	-
EMP Emitter	Techmod	none yet	EMP Pulse (disable enemy auto-weapons)
Interceptor Bay	Fighter bay	none yet (fighter stats are stored)	launches fighters that shoot missiles
Strike Bay	Fighter bay	none yet	launches bombers
Gunner	Crew	+15% fire rate on all turrets	Overcharge
Engineer	Crew	techmod effects +25% ("potency")	Emergency Repair

"Shell" means the item exists, has stats and shows in the UI, but its active ability or fighters do nothing in combat yet. That arrives with M4.

What a drop can roll

ITEM KIND	POSSIBLE STAT LINES (RANGE FOR A RARE ROLL)
Turret weapons	damage +8-20%, fire rate +8-20%, range +10-25%, projectile speed +10-25% (all on that weapon only)
Techmods	shield +10-25, shield regen +1-3, armor +1-2, hull +15-40 (whole ship)
Crew	turret fire rate +3-6% (all turrets), thrust +4-8%, turn rate +5-10%, shield regen +1-2
Fighter bays	hull +10-30, shield +8-20, armor +0.5-1.5

Rarity decides how many lines: Common 0, Rare 1, Epic 2 (Epic values are multiplied by 1.5). Legendary exists as a name but is reserved for later named items.

The three guards of the data layer

- **ContentDB** (`scripts/autoload/content_db.gd`) scans `res://data/` at startup, files every resource by its `id` , and exposes lookups: `get_by_id` , `get_hull` , `hulls()` , `weapons()` , `items()` , `enemies()` .
- **ContentValidator** (`scripts/systems/content_validator.gd`) checks everything the ContentDB loaded and prints problems in debug builds: duplicate ids, a hull whose hardpoint marker is missing from its model, a weapon with no projectile, an enemy carrying a Large weapon in a Small slot, phases out of order, an act with no encounters.
- **Tests** load the real `data/` folder and assert it is clean, so a typo in a `.tres` fails the test run, not your playtest.

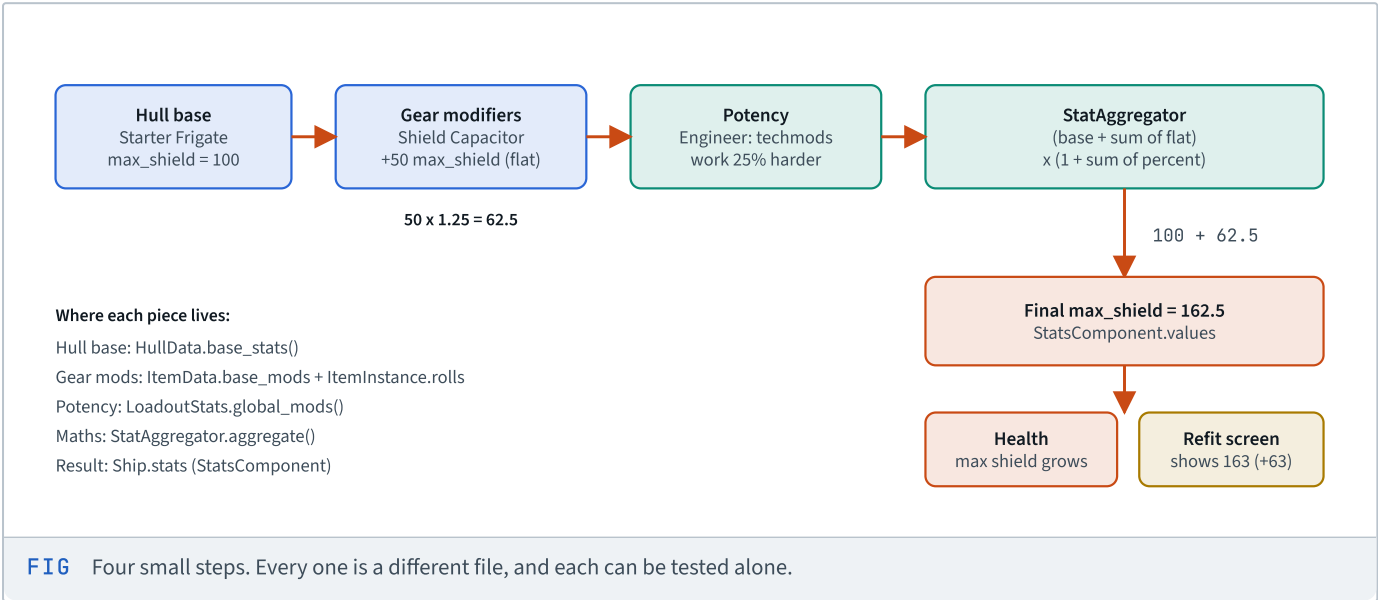
THE GOLDEN RULE OF DATA

Authored resources are **never modified while the game runs**. Tuning panels and rolled stats work on copies or on instances. If you ever see code writing into a `HullData` or `WeaponData` , that is a bug.

Stats: how numbers get combined

A ship's shield, speed or a weapon's damage is never typed in one place. It is built from layers. Understanding this one pipeline explains the hull files, the item rolls, the crew bonuses and the refit screen's green and red numbers.

The pipeline



A modifier, and its two dials

Every bonus anywhere in the game is a `StatMod` : *which stat, how it applies (op) and to what (scope)*, with a value.

DIAL	CHOICES	MEANING
op	FLAT	adds a fixed amount: +50 shield
	PERCENT	scales: +0.25 means +25%. Percent bonuses add together (two +25% make +50%, not +56%)
scope	THIS_ITEM	only the item carrying it (a weapon's own rolled +20% damage)
	ALL_TURRETS	every weapon on the ship (the Gunner's fire-rate bonus)
	SHIP	the ship's own stats (shield, armor, speed); also applies to every weapon's stats

A REAL BUG THIS DESIGN TAUGHT US

A *percent* bonus on a stat that starts at zero stays zero, because zero times anything is zero. The Engineer's "techmod potency" starts at 0, so it had to be a *flat* +0.25 that the UI displays as "+25%". Rule of thumb: percent only for stats that already have a base value.

Two sets of stats: the ship and each weapon

	SHIP STATS	WEAPON STATS
Keys	mass, thrust, max_speed, turn_rate, drag, lateral_grip, burn_impulse, burn_cooldown, max_hull, max_shield, shield_regen, shield_delay, armor, techmod_potency	damage, fire_rate, range, projectile_speed

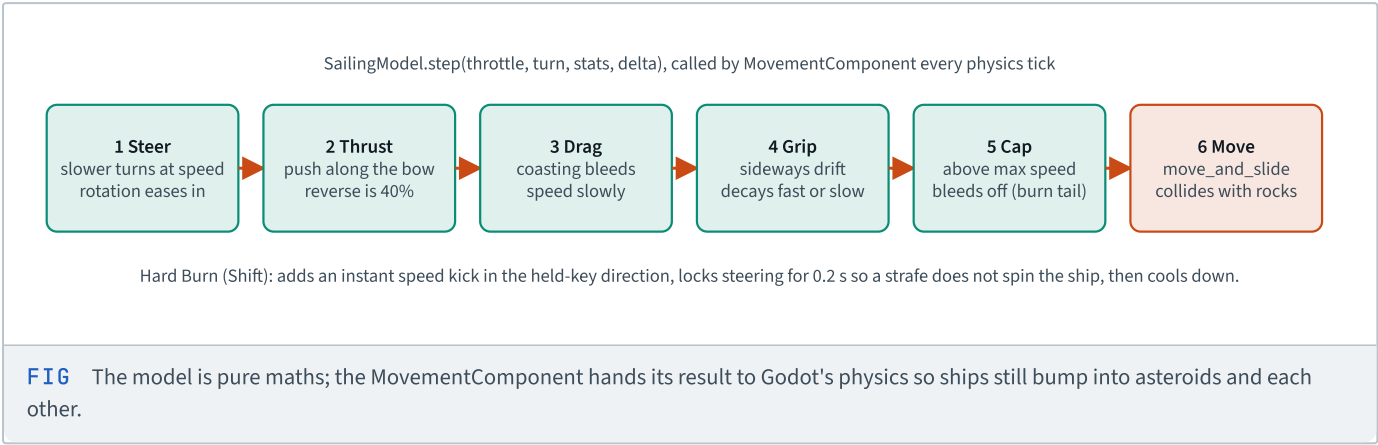
	SHIP STATS	WEAPON STATS
Base comes from	the HullData	the WeaponData and its ProjectileData
Mods that apply	all <code>SHIP</code> -scope mods from everything mounted	the weapon's own <code>THIS_ITEM</code> mods, plus every <code>ALL_TURRETS</code> and <code>SHIP</code> mod
Computed by	<code>StatsComponent</code> (via <code>StatAggregator</code>)	<code>WeaponStats.compute()</code> , stored in each <code>WeaponController.stats</code>
Recomputed when	any gear changes (<code>Ship._apply_loadout</code>)	the same moment

Fire rate is stored as shots per second (cooldown is its inverse), so "+15% fire rate" is a plain percent bonus. A weapon with a longer range also gets proportionally longer-lived shots, so the extra range is real.

`LoadoutStats.global_mods()` is where gear is turned into the modifier list. It also keeps one weapon's rolled bonus from leaking onto another weapon, and applies the Engineer's potency to techmod items only.

Sailing: how a ship moves

The feel you called "very nice" lives in one small pure class. It has no nodes; give it commands and stats and it tells you where the ship goes.



The knobs, and where they live

Knob	What it does	Where to change it
thrust, mass	acceleration = thrust / mass	hull .tres (or F1 panel to try values live)
max_speed	speed cap (burns can exceed it briefly)	hull .tres
turn_rate	degrees per second when still; falls to 40% at full speed	hull .tres ; the 40% is FULL_SPEED_TURN_FACTOR in sailing_model.gd
drag	how quickly a coasting ship slows	hull .tres
lateral_grip	the feel knob: low = sailing slide, high = car-like grip	hull .tres
burn_impulse, burn_cooldown	Hard Burn strength and cooldown	hull .tres
reverse 40%, steering ease-in, 0.2 s burn lock	fixed feel constants	top of sailing_model.gd

TUNING WORKFLOW

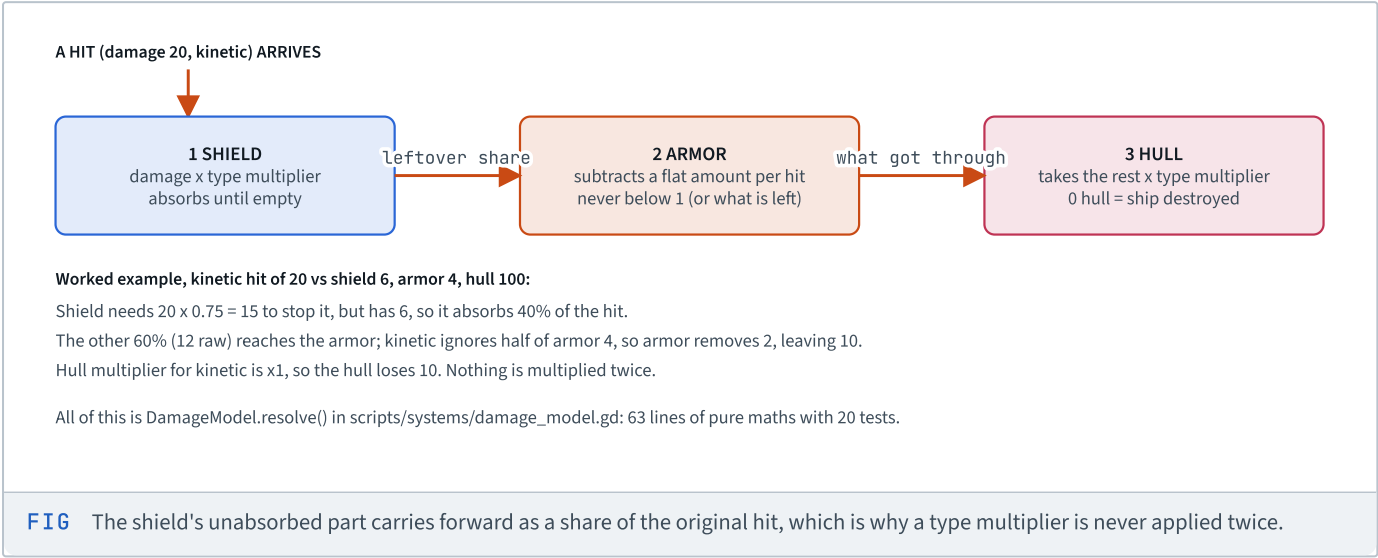
In the sandbox press **F1** for live sliders on the player's hull. Fly around until it feels right, press *Print values* (copied to the clipboard), paste them into the hull's `.tres`. The slider changes never touch the saved file.

Steering convention to remember when reading the code: the ship's nose points toward **-Z**; turning left increases the yaw angle; a turn input of +1 means starboard (right).

Combat: from trigger to damage

Three ideas: weapons decide *when* to fire, projectiles decide *what they hit*, and the damage model decides *what that does*.

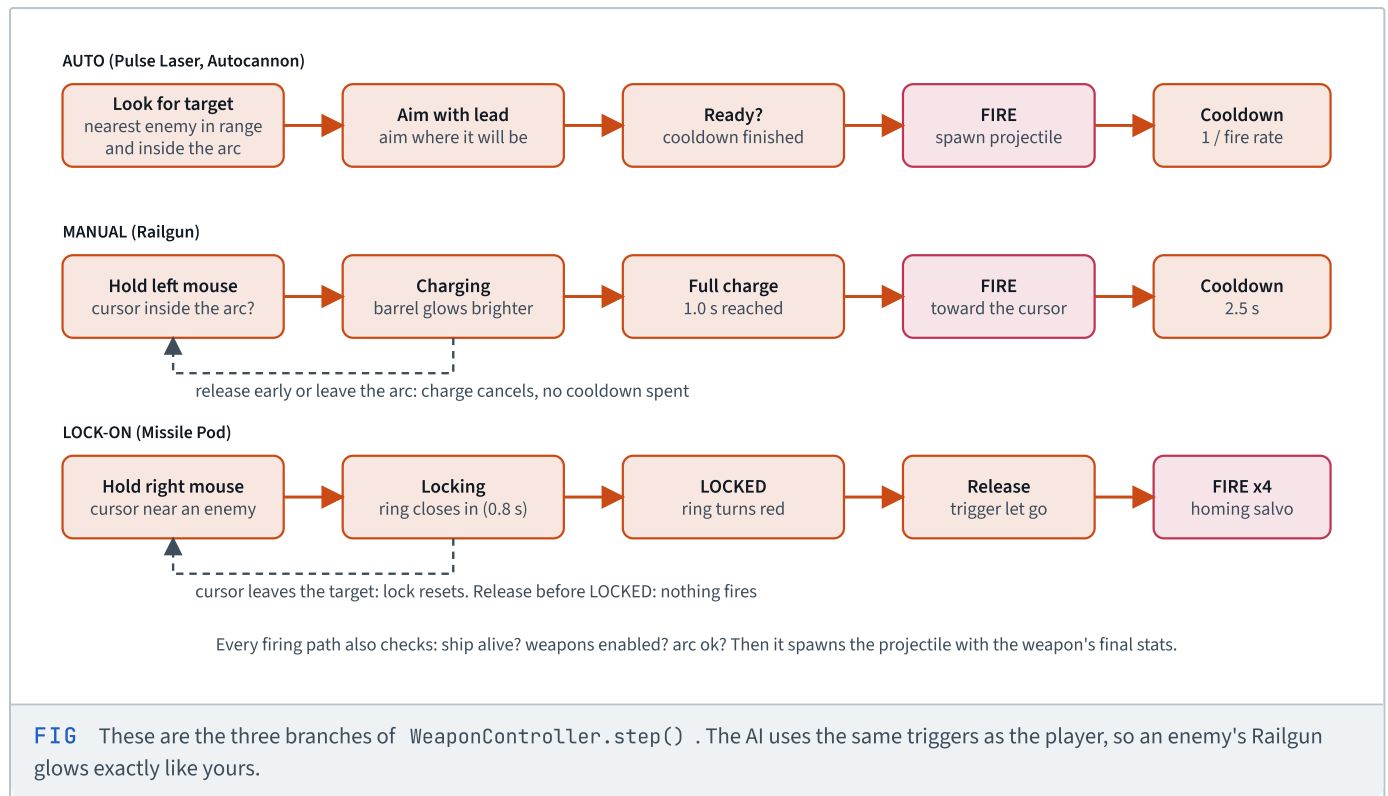
Damage: shields, then armor, then hull



DAMAGE TYPE	VS SHIELD	VS ARMOR	VS HULL	USED BY
Energy	$\times 1.5$	normal	$\times 0.75$	Pulse Laser
Kinetic	$\times 0.75$	ignores 50% of armor	$\times 1$	Autocannon, Railgun
Explosive	$\times 0.5$	normal	$\times 1.5$	Missile Pod

The design intent: energy strips shields quickly but struggles against armored hulls, kinetic punches through armor, explosive finishes hulls. Armor is flat, so many small hits suffer and few big hits don't. That is why the Pulse Laser is weak against the armored Warden. The table lives at the top of `damage_model.gd` and is the lever for global balance.

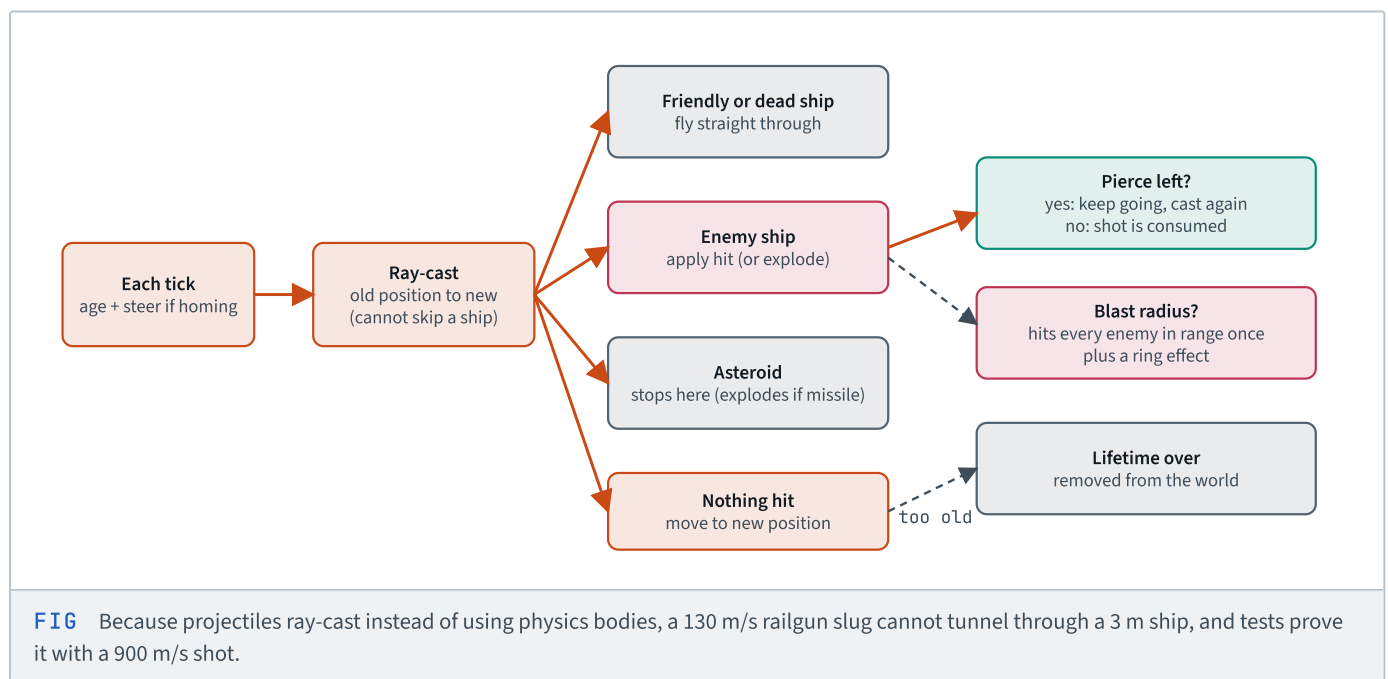
Weapons: three ways to fire



Arcs. Each hardpoint has an arc (360° all-round, 120° forward, 90° broadside). The maths is in `scripts/systems/arc.gd`. Turn on *Show firing arcs* in the F2 panel to see them as coloured wedges.

Lead. Auto weapons use first-order prediction: they aim at where a straight-moving target will be. You can dodge by changing velocity or Hard Burning. Manual weapons shoot exactly at the cursor; lock-on missiles home.

Projectiles: the life of a shot



Projectiles are spawned into a node in the group `projectile_root` (the scene provides it), never as children of the firing ship, so destroying a ship doesn't delete its bullets in flight.

Inventory and refit

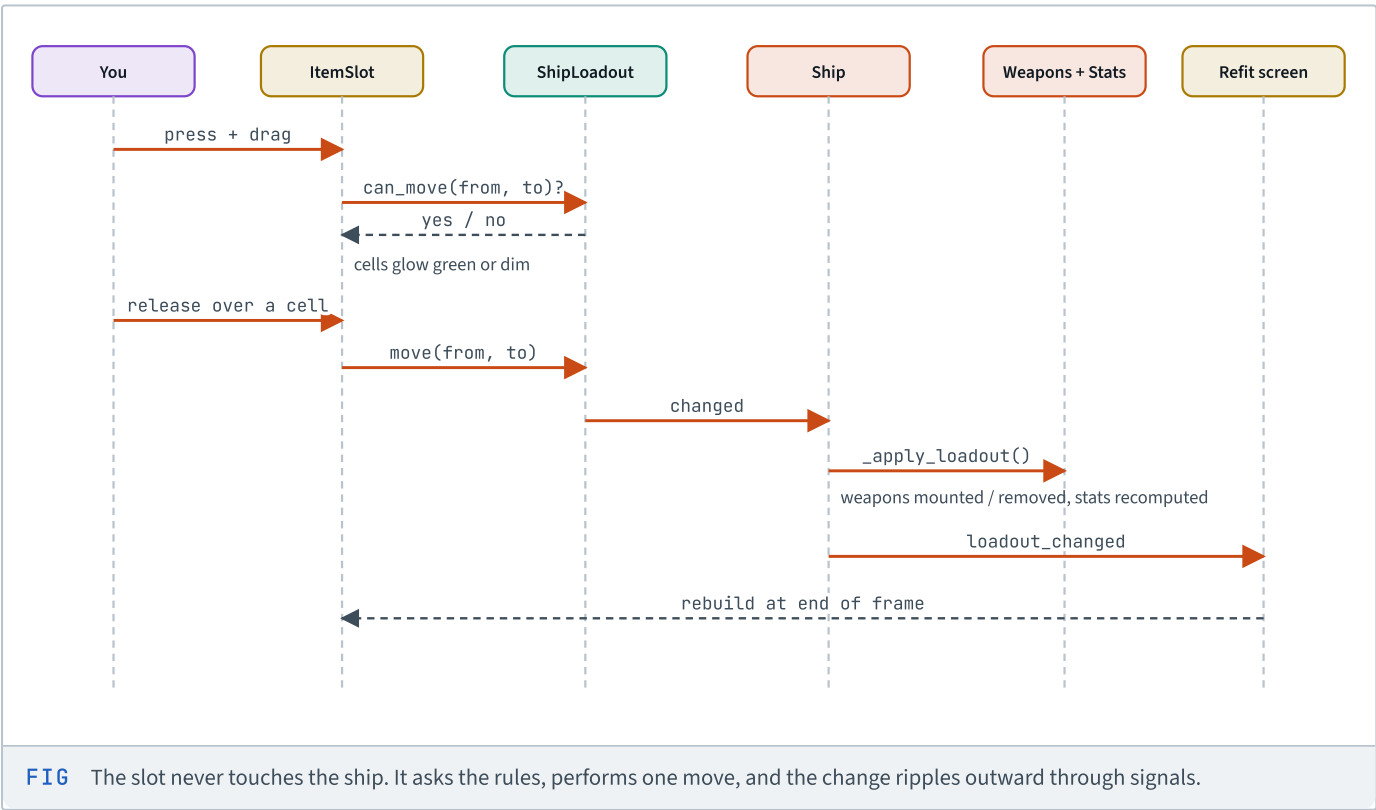
Customization is built from three layers: the rules (ShipLoadout), the ship's reaction (Ship._apply_loadout) and the screen (InventoryScreen). The screen never changes the ship directly; it only asks the rules.

ShipLoadout: the rulebook of what goes where

ShipLoadout (scripts/systems/ship_loadout.gd) is a plain object, not a node. It holds two lists: **slots** (one per hull hardpoint, in the same order) and **cargo** (a fixed number of cells, empty ones are `null` so every item keeps its position). Everything the UI needs is a question or a command on this one object:

METHOD	ANSWERS / DOES
<code>can_accept(kind, index, item)</code>	would this item be legal in that cell? Turrets need a hardpoint of the right type and size (a Small weapon fits a Large turret, not the reverse)
<code>can_move(from, to)</code>	can the item in one cell go to another? If the target is occupied it is a <i>swap</i> , so both items must fit their new homes
<code>move(from, to)</code>	does it, then emits <code>changed</code> . Returns false and changes nothing if illegal
<code>add_to_cargo(item)</code> , <code>equip_auto(item)</code>	first free cargo cell; first compatible empty hardpoint
<code>take(kind, index)</code>	remove and return an item (the discard bin uses it)
<code>rebind_hull(new_hull)</code>	switch hulls: items that fit re-mount, the rest go to cargo, anything with no room is lost
<code>to_dict()</code> / <code>from_dict()</code>	save format. Unknown item ids are skipped; mounted items that no longer fit move to cargo

What happens when you drag an item

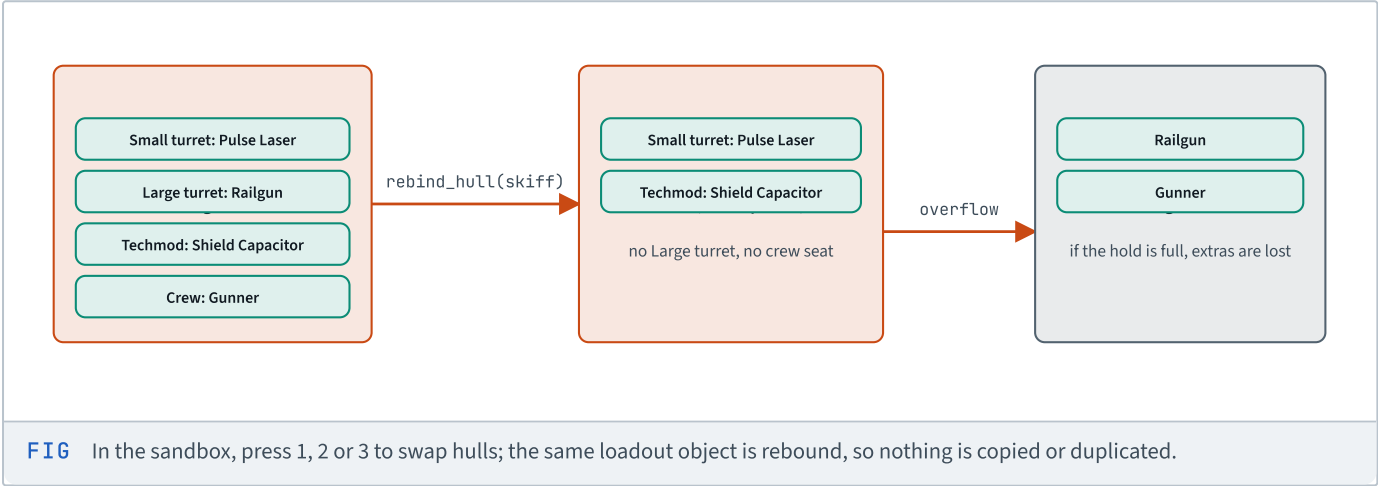


What Ship._apply_loadout does

1. For every hardpoint: if a weapon item is in it and no matching WeaponController exists, make one (a new mount visual + controller). If the slot holds a non-weapon or is empty, remove the controller.
2. Recompute the ship's stats from every equipped item (`LoadoutStats.global_mods`). Health reacts to the new maximums.
3. Recompute each weapon's own stats (own rolls + ship-wide bonuses).
4. Emit `loadout_changed` so the HUD and refit screen redraw.

Because one function does this for every change (drag, auto-equip, sandbox spawn, boss phase adding a weapon), there is only one place to get it wrong, and the tests cover it.

Switching hulls keeps your gear



The screen: five small UI classes

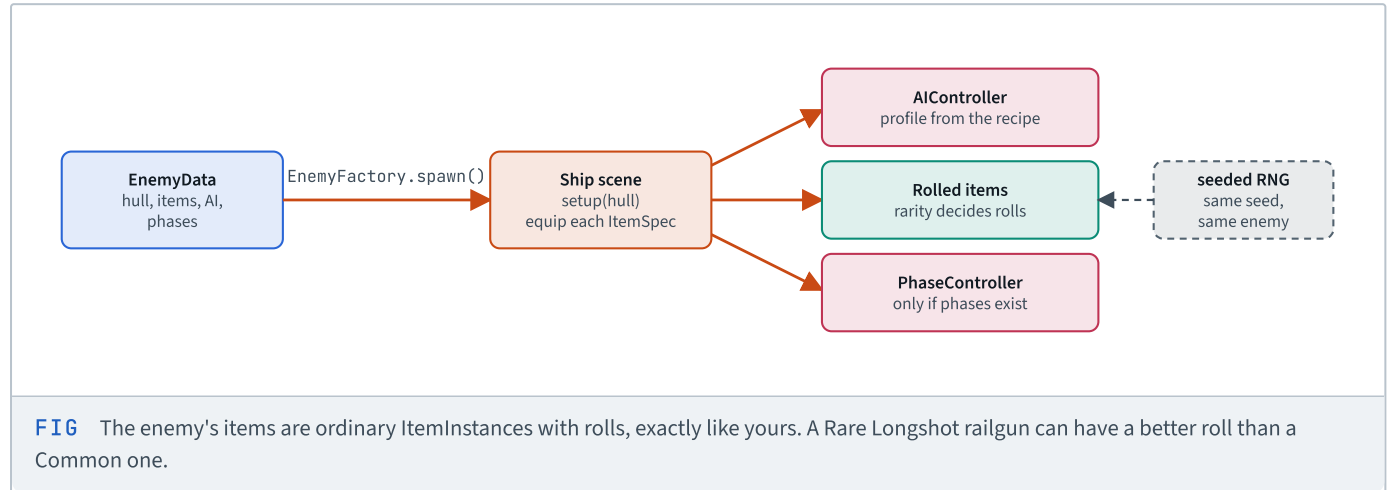
FILE	JOB
<code>inventory_screen.gd</code>	builds the three columns (ship slots, cargo grid, stats), opens with I, closes with I or Esc, pauses the game while open, redraws on <code>loadout_changed</code>
<code>item_slot.gd</code>	one cell: draws the item, starts a drag, answers "can I drop here?" by asking the loadout, highlights valid targets while dragging; also serves as the discard bin
<code>item_tooltip.gd</code>	the hover card: name in rarity colour, numbers, rolled bonuses, description, active ability
<code>stats_panel.gd</code>	the right column: final stats with green/red change from the bare hull, plus DPS per weapon
<code>stat_info.gd</code> , <code>item_visuals.gd</code>	pure presentation: stat names, number formats, colours, two-letter badges

PLACEHOLDER ART, REAL STRUCTURE
Items show as a coloured square with two letters and a rarity border. When real icons arrive, only `item_slot.gd` and `item_visuals.gd` change.

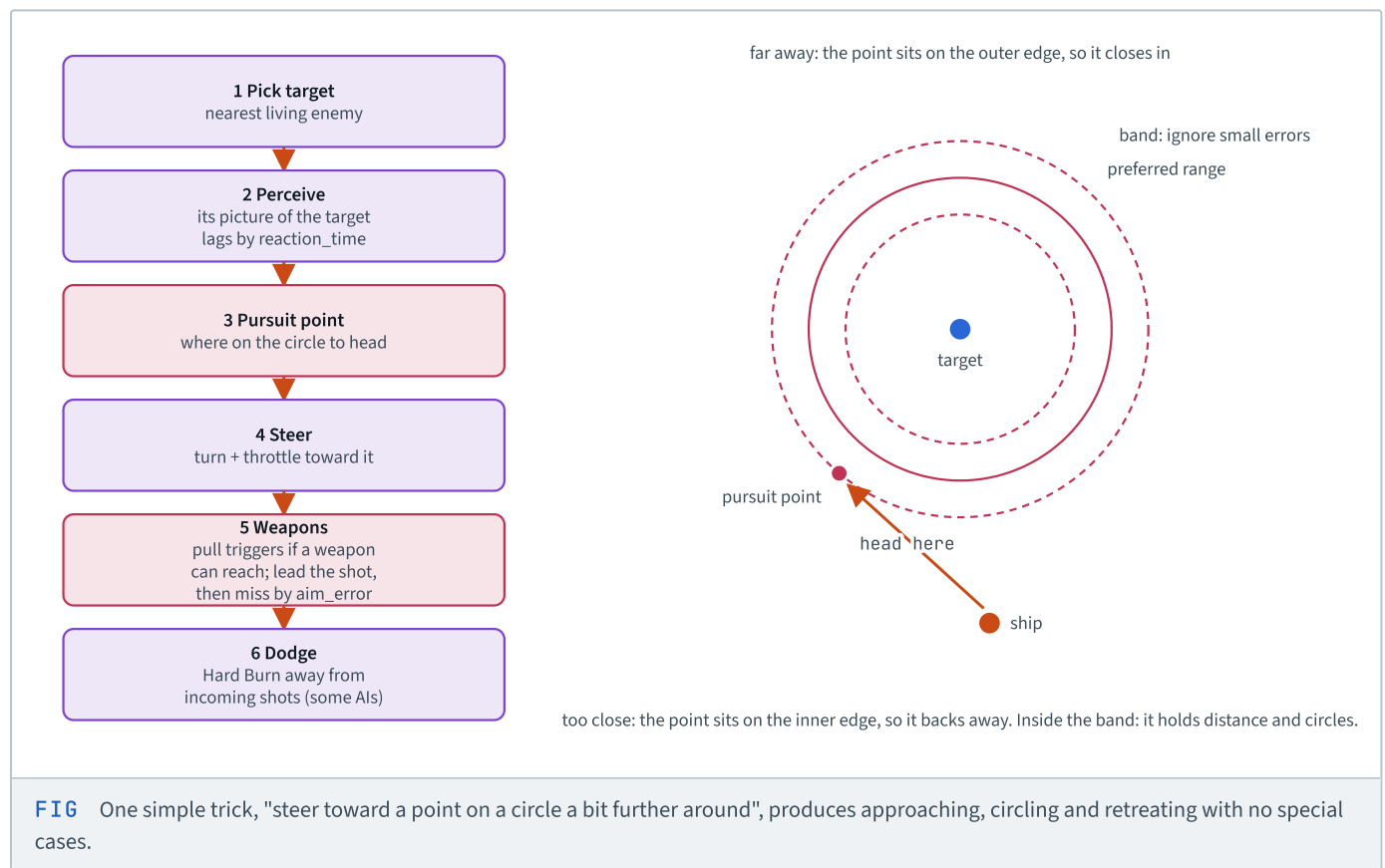
Enemy AI and bosses

An enemy is a recipe plus a brain. The recipe says what it is; the AIProfile says how it fights; the AIController does the fighting using the same orders as you.

How an enemy is born



The AI's loop, every physics tick

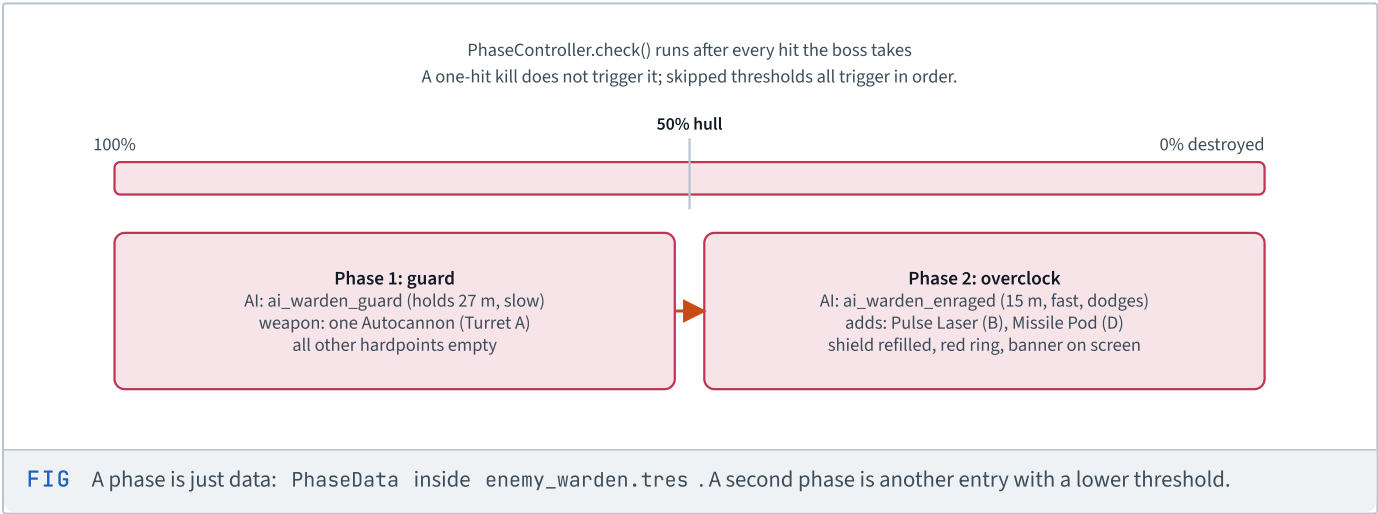


Fighting style is four numbers

PROFILE	RANGE (BAND)	CIRCLING	REACTION	AIM ERROR	DODGES	USED BY
ai_brawler	16 m (+/-5)	40° lead, flips every 3-6 s, 90% throttle	0.4 s	7°	no	Raider
ai_sniper	36 m (+/-6)	22° lead, flips every 5-9 s, 80% throttle	0.45 s	2.5°	yes	Longshot
ai_warden_guard	27 m (+/-6)	28° lead, 60% throttle	0.5 s	5°	no	Warden, phase 1
ai_warden_enraged	15 m (+/-5)	45° lead, flips every 2.5-5 s, full throttle	0.3 s	4°	yes	Warden, phase 2

- **Auto weapons need no AI help.** They target and lead by themselves (they are "auto" for you too), so they are accurate. The profile's aim error only blurs *manual* and *lock-on* shots.
- **Triggers.** The AI holds the manual trigger when its Railgun can reach you (you see the glow). For lock-on it holds the trigger until the lock completes, then lets go so the salvo fires. (A bug in the first version held it forever; a test now guards it.)
- **Dodging.** If a hostile projectile will pass within 1.4 ship-radii within 0.7 seconds, and Hard Burn is ready, it bursts sideways. Brawlers don't; snipers and the enraged Warden do.
- **Disabled AI.** `enabled = false` parks the ship. Duels use it for the intro preview.

The Warden's phase change



BALANCE WAS MEASURED, NOT GUESSED

`tests/probe/probe_balance.gd` makes an AI-driven frigate with the starter loadout fight each enemy. Today: Raider about 75-95 s, Longshot 55-80 s, Warden 70-80 s (the bot loses to the Warden after getting it to about one third). The spec asks for 1-3 minutes per duel. The probe is the tool to re-run after any tuning.

Game flow: scenes, runs and duels

How you get from the sandbox into a fight and back, who remembers what in between, and how pausing works.

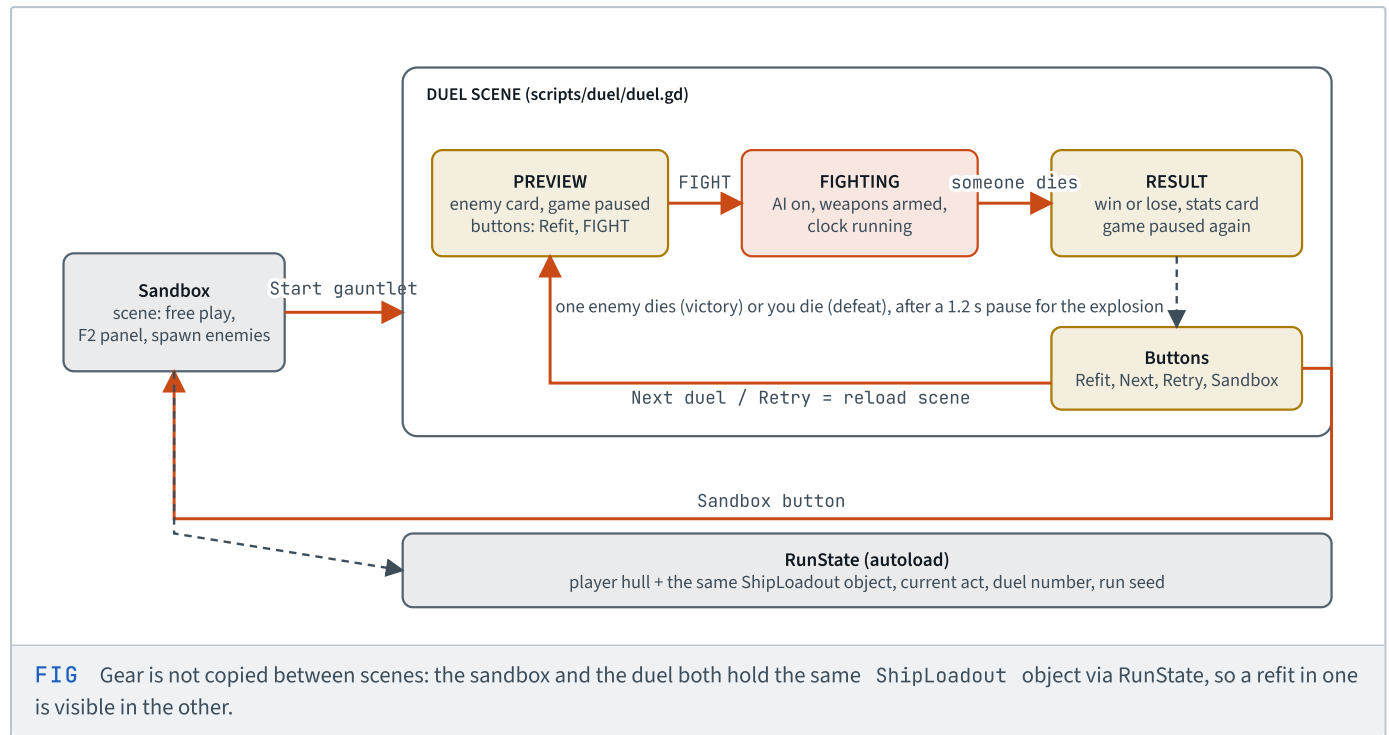


FIG Gear is not copied between scenes: the sandbox and the duel both hold the same `ShipLoadout` object via `RunState`, so a refit in one is visible in the other.

RunState: the run's memory

`player_hull, loadout` what you fly and carry. Created on demand as a Starter Frigate with Pulse Laser + Railgun
`act, duel_index` which enemies are in the run and which one you are on (`current_enemy()` , `has_next()` , `advance()`)
`seed_value, make_rng()` a fresh random generator derived from seed + duel number, so Retry fights the same enemy with the same rolls
`last_result` the stats of the most recent duel

What it does *not* have yet: saving to disk, hull damage carrying between duels, scrap, rewards. Those are M4. The save format for items and loadouts already exists (`to_dict`); only the file writing is missing.

How a duel is assembled

1. `Duel._ready()` asks `RunState` for the player's hull and loadout, and for the current enemy.
2. It builds the player ship (`setup(hull, [], loadout)`) at one end of the arena, adds `PlayerInput` , and disarms it.
3. `EnemyFactory.spawn()` builds the enemy at the other end; its AI is switched off and its weapons disarmed.
4. It adds the HUD, refit screen (`InventoryScreen`), `DuelUI` , damage numbers, and subscribes to `EventBus` for damage tallies and boss banners.
5. It shows the preview and pauses the game. **FIGHT** arms everyone, switches the AI on and unpauses.
6. When either ship dies (`died` signal) the duel records a `DuelResult` , waits 1.2 s, shows the result card and pauses again.

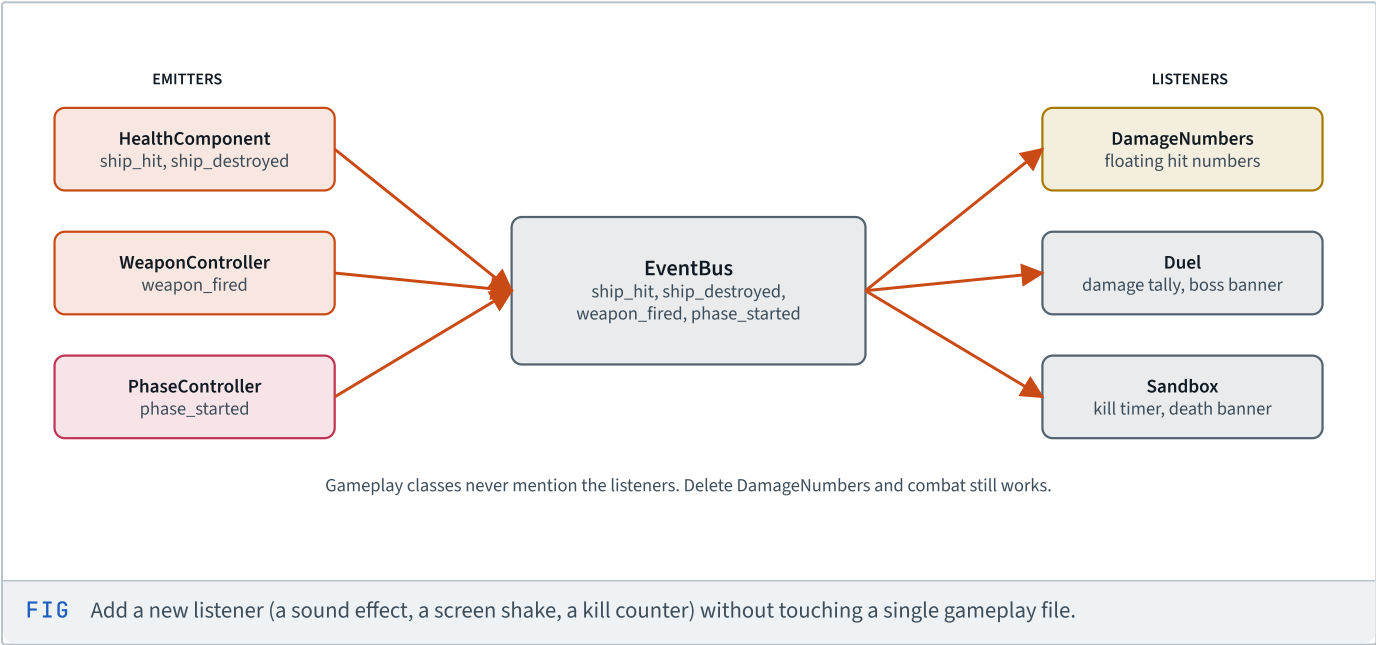
HOW PAUSING WORKS (THE ONE SLIGHTLY FIDDLY SPOT)

Overlays and the refit screen use process mode *Always* so they keep running while the game tree is paused. The refit screen unpauses the game when it closes, so the duel listens for its `closed` signal and re-pauses if an overlay is still showing. Tests set `duel.pause_enabled = false` so a paused tree doesn't freeze the test runner.

How the parts talk to each other

Five ways, used consistently. If you know which way a piece of code uses, you know where to look for the other end.

WAY	DIRECTION	EXAMPLE	WHY THIS WAY
Direct method call	downward (owner to part)	<code>ship.health.apply_hit(hit)</code>	the caller owns or knows the target
Command API	controller to Ship	<code>ship.set_throttle(1.0)</code>	same five orders for player, AI and tests
Signal	part to owner / listeners	<code>health.destroyed</code> , <code>loadout.changed</code>	the sender doesn't know who listens
EventBus	anywhere to anywhere	<code>EventBus.ship_hit</code>	decouples gameplay from feedback screens
Groups	find things in the world	<code>get_nodes_in_group("ships")</code>	no stored references to things that may be freed



Every signal in the project

SENDER	SIGNAL	CARRIES	LISTENED TO BY
Ship	<code>hull_changed</code>	the new hull	HitFlash, tuning panel, refit screen, sandbox panel
Ship	<code>loadout_changed</code>	-	HUD (weapon list), refit screen
Ship	<code>burn_used</code>	-	(available for effects)
Ship	<code>died</code>	the killer	Duel, Sandbox (dummy respawn)
HealthComponent	<code>damaged</code>	a DamageResult	HitFlash, PhaseController
HealthComponent	<code>destroyed</code>	the killer	Ship (starts death)
StatsComponent	<code>stats_changed</code>	-	HealthComponent
ShipLoadout	<code>changed</code>	-	Ship (<code>_apply_loadout</code>)

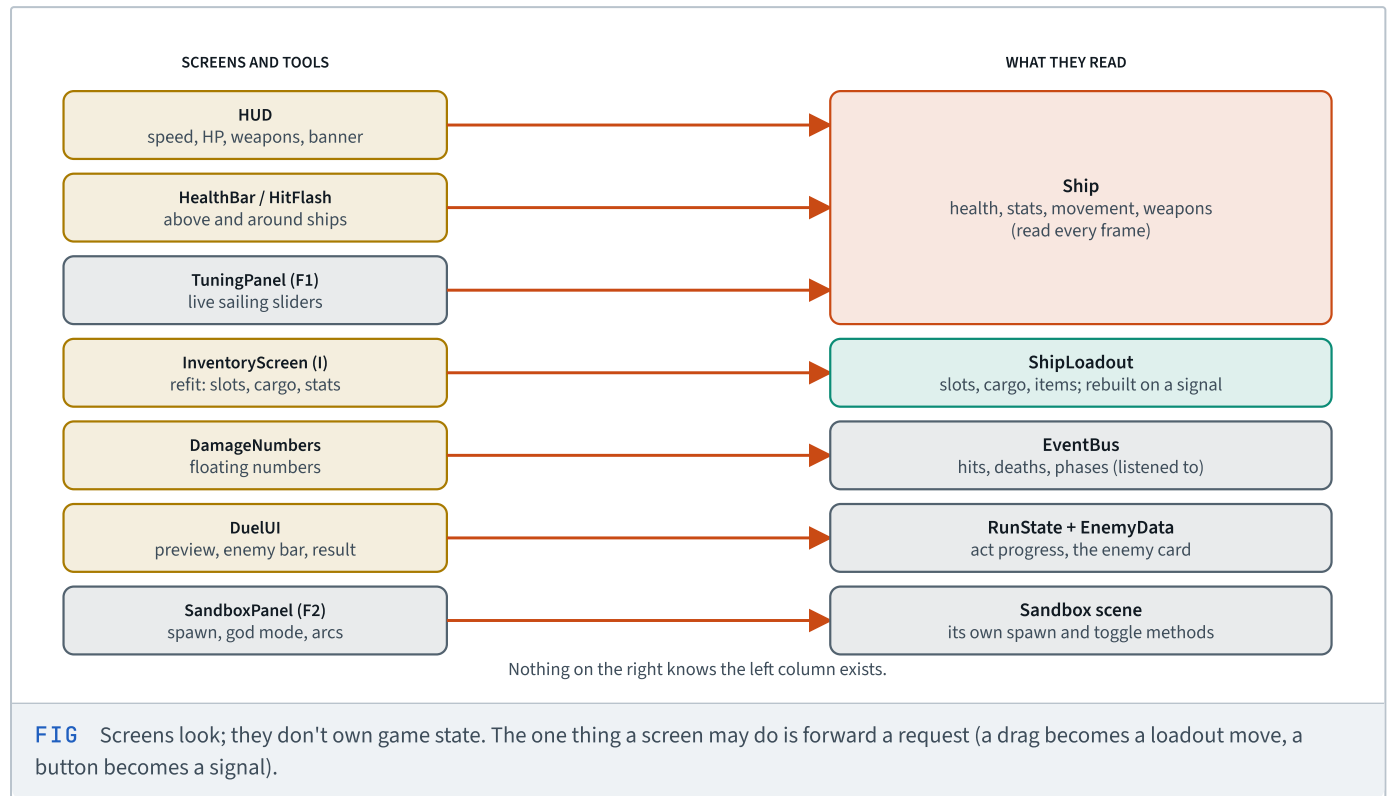
SENDER	SIGNAL	CARRIES	LISTENED TO BY
WeaponController	fired	-	AIController (re-rolls aim error)
PhaseController	phase_started	index, PhaseData	(also sent on EventBus)
InventoryScreen	closed	-	Duel (re-pause)
DuelUI	fight_pressed , refit_pressed , next_pressed , retry_pressed , menu_pressed	-	Duel

Groups and global names

- ships : every Ship (so "find my enemies" is one call). projectiles : every live shot (AI dodging scans it).
projectile_root : the node new shots are added under.
- Autoloads are visible everywhere by name: ContentDB , RunState , EventBus .
- Teams are plain names on a ship: player and enemy . Targeting, projectiles and AI all compare these.

Screens and tools

Everything you see on top of the 3D world. All of it is built in code (no hand-made UI scenes yet), which keeps changes visible in one file but means layout tweaks are code edits.



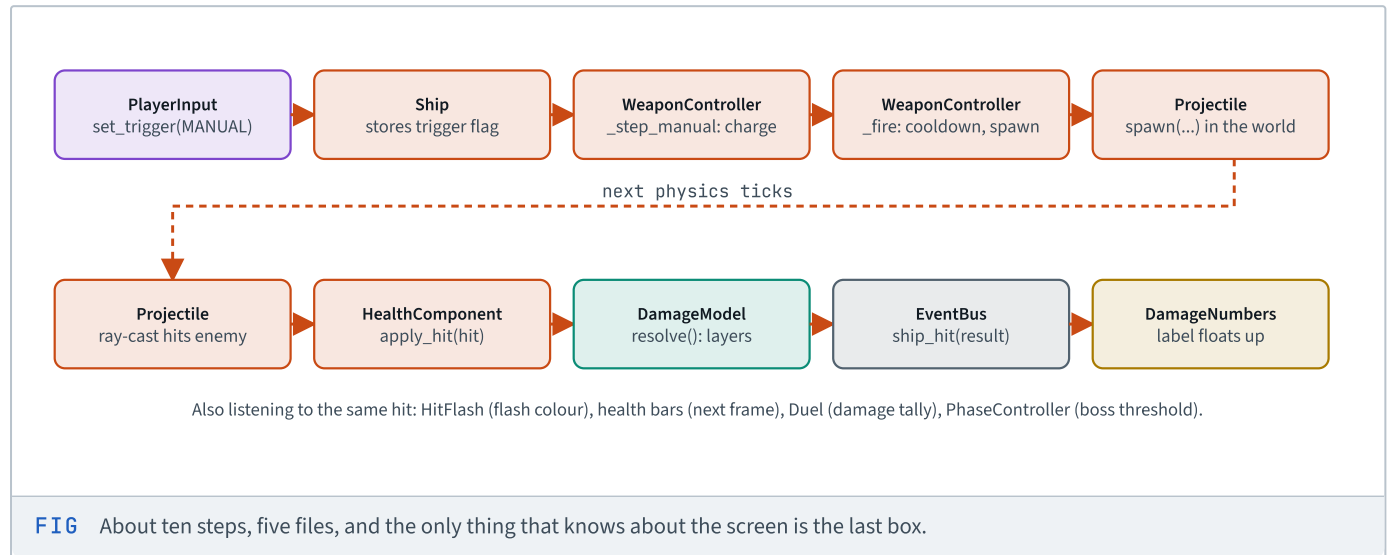
Hotkey and tool cheat sheet

KEY	WHERE	DOES
W / S, A / D	everywhere	thrust / reverse, steer
Shift (+ W/A/S/D)	everywhere	Hard Burn; the held direction picks the way (none or W = forward, S = back, A/D = strafe)
Mouse; left button; right button	everywhere	aim; manual weapons; hold on an enemy to lock, release to fire
I (or Esc to close)	sandbox, duel	refit screen (pauses the game)
F1	sandbox	tuning panel: live sliders for the player's hull stats; <i>Print values</i> copies them
F2	sandbox	sandbox panel: hull picker, add items (with rarity), spawn enemy/dummy/bot, start gauntlet, god mode, bots fire, show arcs
1-9	sandbox	switch hull (gear follows)
B, N, X, R	sandbox	spawn bot, spawn dummy, clear enemies, revive player
[] and \	sandbox	halve / double / reset time scale
Mouse wheel	sandbox	camera zoom

Three guided walks through the code

Reading a map is one thing; walking a route is another. Each walk follows one thing end to end and names the file and function at every step.

Walk 1: you click, and a number floats off an enemy



1. `player_input.gd` `_physics_process` : reads the left button and tells the ship the manual trigger is held.
2. `weapon_controller.gd` `_step_manual` : if the cursor is inside the arc and the weapon is ready, builds charge; at full charge calls `_fire` .
3. `_fire` sets the cooldown (1 / fire rate), spawns the first projectile (more if it is a salvo), emits `fired` and `EventBus.weapon_fired` .
4. `projectile.gd` `_physics_process` : moves, ray-casts, finds an enemy ship, calls `health.apply_hit(Hit)` (or explodes).
5. `health_component.gd` `apply_hit` : asks `DamageModel.resolve` , stores the new shield and hull, emits `damaged` and `EventBus.ship_hit` ; if hull is gone it emits `destroyed` .
6. `damage_numbers.gd` hears `ship_hit` and spawns blue, grey and red labels from the `DamageResult` .

Walk 2: you drag a Shield Capacitor onto the techmod slot

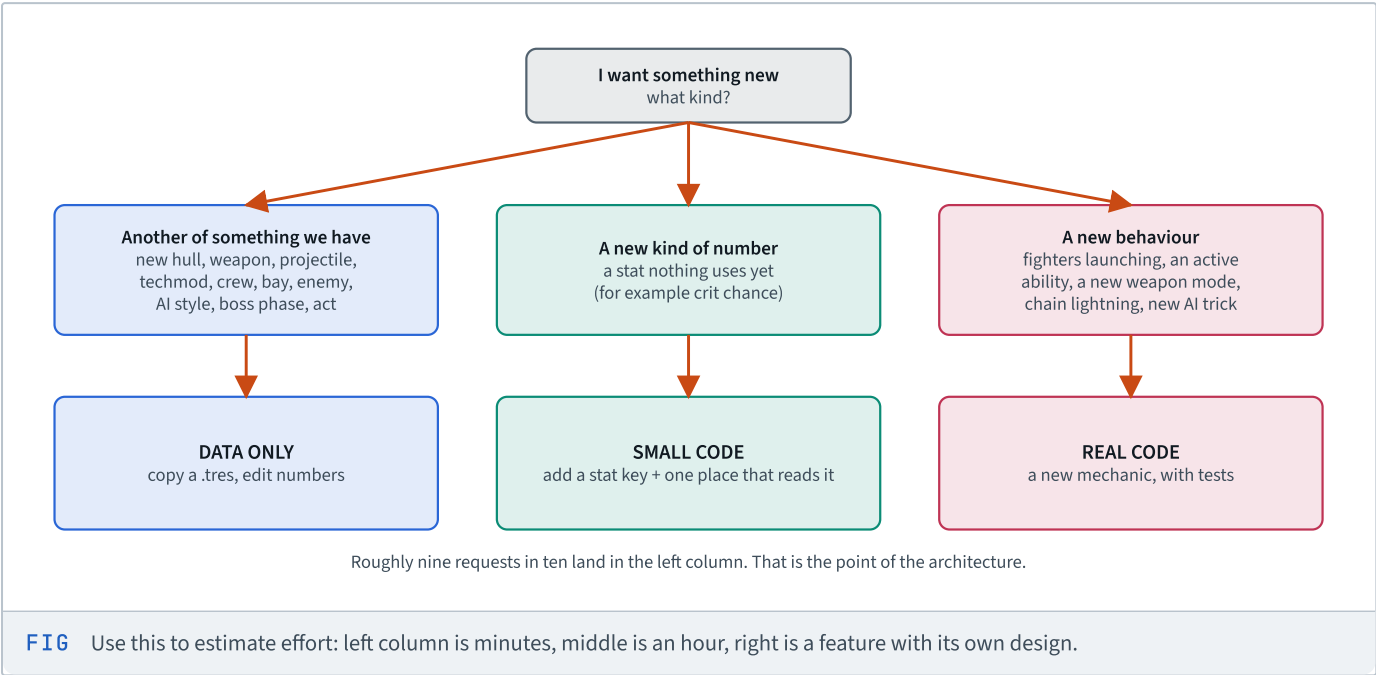
1. `item_slot.gd` `_get_drag_data` : begins the drag, returns where it came from. All slots call `_notification` and highlight if `loadout.can_move(...)` .
2. On drop, `_drop_data` calls `loadout.move(CARGO, i, SLOT, 3)` .
3. `ship_loadout.gd` checks the capacitor is a techmod and slot 3 is a techmod hardpoint, swaps the cells, emits `changed` .
4. `ship.gd` `_apply_loadout` : asks `LoadoutStats.global_mods` for the +50 shield / +4 regen, sets them on `StatsComponent` (which emits `stats_changed`).
5. `health_component.gd` `_on_stats_changed` : the maximum grew by 50, so the current shield grows by 50 too.
6. `ship.gd` emits `loadout_changed` ; `inventory_screen.gd` redraws at end of frame and `stats_panel.gd` shows *Shield 150 (+50)* in green.

Walk 3: you press Start gauntlet and the Raider appears

1. `sandbox_panel.gd` button calls `sandbox.start_gauntlet()` , which calls `RunState.start_act(slice_act)` (new seed, duel 0) and changes scene to `duel.tscn` .
2. `duel.gd` `_ready` : reads the enemy from `RunState` (`enemy_raider.tres`) and builds your ship from `RunState`'s loadout.
3. `enemy_factory.gd` `spawn` : instantiates `ship.tscn` , sets team `enemy` , `setup(raider hull)` , rolls each `ItemSpec` with `ItemRoller` from `RunState.make_rng()` and equips it, then adds an `AIController` using `ai_brawler.tres` .
4. `hardpoint_mount.gd` builds the barrel; `weapon_controller.gd` is created for the autocannon; `health_bar.gd` floats the bar overhead.
5. `duel_ui.gd` shows the preview card from the same `EnemyData` (hull numbers, style text, weapons coloured by rarity). The tree is paused.
6. You press FIGHT: `Duel.begin_fight()` arms both ships, enables the AI and unpauses.

How to extend the game

The most useful page for a client who wants to direct the work. Before asking for a feature, ask this question: does it need new code, or only a new file?



Recipes

I WANT TO ADD...	DO THIS	CODE?
A weapon that behaves like the existing ones	Copy <code>data/projectiles/pulse_bolt.tres</code> and <code>data/weapons/pulse_laser.tres</code> ; change id, name, numbers; point the weapon at the new projectile. It shows up in the F2 item list automatically.	no
A hull	Make a model scene in <code>scenes/ship/models/</code> with a <code>Marker3D</code> for every hardpoint; copy a hull <code>.tres</code> ; list the hardpoints (type, size, arc, marker name). Set <code>player_selectable</code> false for enemy-only hulls.	no
A techmod or crew member	Copy an existing one. <code>base_mods</code> is the list of StatMods it gives. Pick a roll pool.	no
An enemy	New <code>EnemyData</code> : hull, a list of ItemSpecs (item + rarity + slot name), an AI profile. Add it to <code>data/acts/slice_act.tres</code> to appear in the gauntlet.	no
A boss phase	Add a <code>PhaseData</code> to the enemy: threshold, new AI profile, extra items, shield refill, banner text. Keep thresholds falling.	no
A different enemy personality	New <code>AIPProfile</code> : range, circling, reaction, aim error, dodging. Only if none of the knobs can express it, change <code>ai_controller.gd</code> .	usually no
A new stat (crit chance, evasion...)	Add a key in <code>stats.gd</code> ; add its base value where the stat's owner is (<code>HullData.base_stats()</code> for ship stats, <code>WeaponStats.base()</code> for weapon stats); add a label in <code>stat_info.gd</code> ; write the one place that reads it; add tests.	small
Fighters, Q/E/R abilities, a new weapon mode, status effects	This is the planned <i>Effect pipeline</i> (spec 5.4): an <code>Effect</code> resource with hooks (on fire, on hit, on kill, on tick) run by an <code>EffectRunner</code> on the ship. The data side (<code>FighterBayData</code> , <code>active_name</code>) already exists. Plan: M4.	real
A new screen or HUD element	New script under <code>scripts/ui/</code> ; read from the ship or listen to <code>EventBus</code> . Never make gameplay call into it.	small

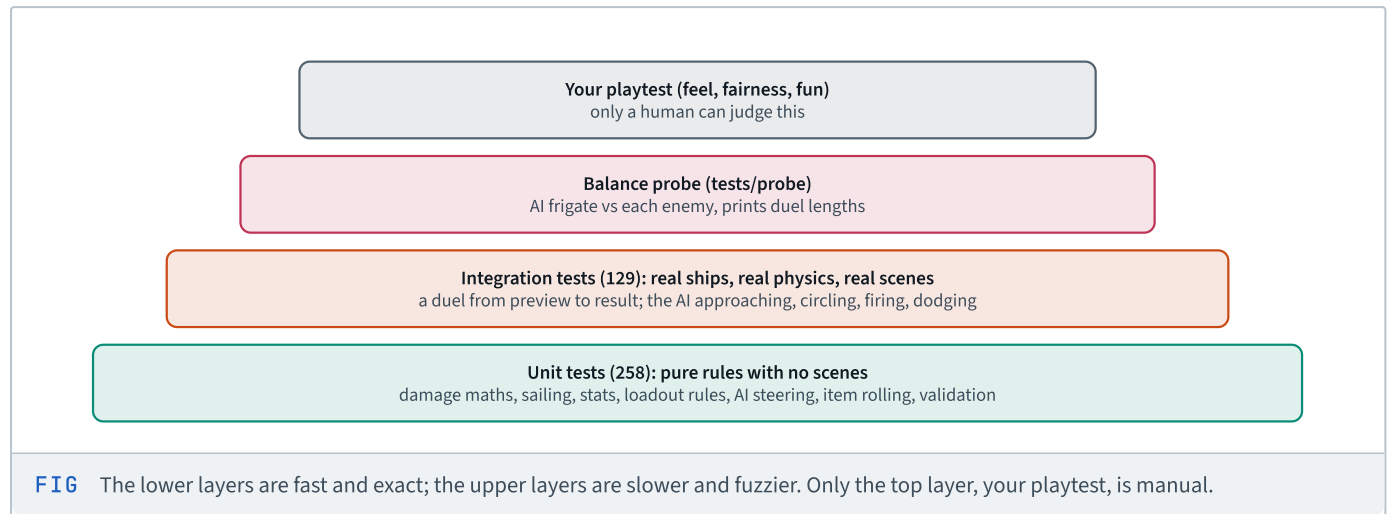
I WANT TO ADD...	DO THIS	CODE?
Real art	Replace model scenes (keep marker names), item badges in <code>item_visuals.gd</code> / <code>item_slot.gd</code> , projectile meshes in <code>projectile.gd</code> , the floor shader. Rules are untouched.	no (art work)

After any change

1. Import (Godot does this on opening the editor; from the command line it is the `--import` step).
2. Run the test suite. The validator test will catch a bad `.tres` (wrong slot, missing marker, duplicate id) with a message that names the file.
3. Open the sandbox and look: F2 → spawn it, fight it, open the refit screen.
4. If it affects balance, run the probe and compare duel lengths.

Testing and tooling

387 automated tests protect the rules, and three habits protect the feel: the balance probe, headless screenshots, and your own playtests.



What is covered

AREA	TESTS	GUARDS
Sailing	27	turn-rate falloff, thrust and drag, grip, Hard Burn directions and cooldown
Damage	20	every multiplier, layer order, overflow, armor minimum, destruction
Weapons	40 (+16 projectile, 10 combat)	cooldowns, arcs, lead maths, charge, lock-on, salvos, seeded spread, projectiles piercing / blocked / homing / blasting / not tunnelling
Health and stats	16 + 12 + 16	regen delay, god mode, stat order, scopes, potency, weapon stats
Inventory	22 + 18 + 18 + 19	every legal and illegal move, saving, hull switch, stats on the ship, the UI's drag callbacks and tooltips
Items and data	18 + 15 + 8 + 8 + 16	rolling, rarity, save round trips, validator errors, the real data folder is clean
Enemies and AI	20 + 20 + 15 + 10	steering maths, factory output, phases, AI behaviours in real physics, run state
Flow	13 + 7 + 11	duel preview/fight/result, sandbox, ship setup

How to run things

```
# the engine (the _console build prints to the terminal)
GODOT = "C:\Users\Kevin Sukias\Downloads\New folder\Godot_v4.7.1-stable_win64_console.exe"

# after adding or renaming any class_name script:
GODOT --headless --path . --import --quit

# all tests (the --fixed-fps flag makes physics ~340x faster; always use it)
GODOT --headless --fixed-fps 60 --path . -s addons/gut/gut_cmdln.gd

# the balance probe
```

```
GODOT --headless --fixed-fps 60 --path . -s addons/gut/gut_cmdln.gd -
gtest=res://tests/probe/probe_balance.gd
```

WHY THE TESTS CAN FIGHT WHOLE DUELS IN SECONDS

Normally a game runs in real time, so a 90-second fight takes 90 seconds. `--fixed-fps 60` tells Godot to advance 1/60 s per step as fast as the computer can. A minute of combat takes a fraction of a second, which is what makes the balance probe and AI tests possible.

How a test is written

- Unit tests extend `GutTest` and call pure functions: `DamageModel.resolve(10, KINETIC, 100, 3, 100)` then assert the numbers.
- Combat tests extend `tests/helpers/ship_test.gd`, which provides `make_ship()`, `weapon()`, `sim()` (steps weapons and projectiles by hand) and cleans up shots after each test.
- Anything a test spawns must be registered for cleanup (`autofree`) or it keeps fighting during the next test. This bit us once.
- Names read as sentences: `test_a_phase_triggers_only_once`. A failure message is the specification that broke.

Conventions and gotchas

The unwritten rules of this codebase, written down. Most of these exist because something went wrong once.

House style

- **Static typing everywhere.** Godot warns on untyped declarations (project setting). Variables, parameters and returns carry types.
- **One class per file, named for the file** (`sailing_model.gd` defines `SailingModel`). Files are snake_case, classes are PascalCase.
- **Constants at the top, with a sentence of why.** Tuning numbers that are not per-item (the damage table, sailing constants) are named constants, not buried in logic.
- **Stat names are constants** in `Stats` , never raw strings. A typo then fails at load time instead of silently doing nothing.
- **Comments explain why**, not what. Public classes start with a `##` description.

Coordinates and units

- The play space is the flat plane $y = 0$. Ships are locked to it; projectiles are flattened to it.
- A ship's **nose points toward -Z**. Yaw increases when turning left. Arc bearings are measured from the nose, positive toward starboard.
- Distances are metres, speeds metres per second, turn rates degrees per second (converted to radians inside the maths).
- Percent values in data are fractions ($0.15 = 15\%$).

Godot-specific traps we have already stepped in

TRAP	WHAT HAPPENS	THE RULE
Freed objects in typed variables	A weapon kept a reference to a ship that was destroyed and freed, then erred every frame passing it to a function	Check <code>is_instance_valid()</code> before using any stored reference to something that can disappear. (Do not rely on <code>≠ null</code> .)
Physics interpolation is on	Ships look smooth between physics ticks. Things moved in <code>_process</code> can jitter; teleports leave a smear	Move things in <code>_physics_process</code> ; call <code>reset_physics_interpolation()</code> after teleporting; UI that follows a ship reads its interpolated transform.
Percent bonus on a base of zero	Zero times 1.25 is zero (the Engineer bug)	Percent only on stats with a base; otherwise store a flat fraction.
Autoloads in command-line scripts	A throwaway <code>-s</code> script can't name classes that use autoloads at compile time	Throwaway scripts wait two frames and use <code>get_node("ContentDB")</code> . The game itself is unaffected.
Adding a class_name	Tests fail with "class_names have not been imported"	Run the import step once after adding or renaming classes.
Hand-editing .tres	Enums are stored as numbers; typed arrays use <code>Array[ExtResource("id")]([...])</code>	Copy an existing file and change values rather than writing from scratch.
Line-ending warnings from git	"CRLF will be replaced by LF" on Windows	Harmless noise; the repository normalises line endings.
<code>.gd.uid</code> files	Godot writes one beside every script	Keep them in git; deleting them breaks references.

Never do these

- Never write into an authored `.tres` at runtime. Make an instance or a copy.
- Never let gameplay code call into a screen. Emit a signal or EventBus event instead.
- Never give a controller a private back door into a ship. If the AI needs something the player doesn't have, add it to the command API for both.
- Never store a ship, projectile or lock target forever without a validity check.
- Never bypass `ShipLoadout` to change what is mounted; the ship would not update.

Architect's health check

An honest assessment of the building as it stands: what is solid, what is a shortcut, and what to prepare before M4 adds the most code.

What is solid

- **Clean layering.** Data, rules, ship parts, scenes and screens depend downward only. The rules layer has no scene dependencies, which is why it is exhaustively tested.
- **One door for control.** Player, AI, debug bots and tests all use the same five commands. That is why enemies were cheap to add.
- **Content scales by files.** Eleven items, seven hulls, four AI styles, three enemies and an act exist without a line of content-specific code.
- **Events decouple feedback.** New effects, sounds and counters can listen without touching gameplay.
- **Tested balance, not just logic.** Duel length is measurable; the probe turned "too lethal" from a feeling into numbers.
- **The unglamorous parts exist.** Content validation, a seeded RNG path, save-ready dictionaries and unknown-id tolerance are in place before they were urgent.

Risks and shortcuts, ranked

#	ISSUE	WHY IT MATTERS	SUGGESTED FIX	EFFORT	WHEN
1	The sandbox is the project's main scene	A player opening the game lands in a debug tool	Add a small main menu scene (Play gauntlet, Sandbox) and make that the main scene	small	start of M4
2	<code>WeaponController</code> (348 lines) does targeting, charging, lock-on, salvos and a UI marker	Any new weapon mode edits the biggest file	Split the three modes into small strategy classes and move the lock marker to a visual child	medium	before adding a 4th mode
3	<code>Ship</code> (264 lines) is the hub of everything	Fine today; fighters and abilities will pull more into it	Keep new features as components (an <code>AbilityController</code>) like the spec says, rather than methods on <code>Ship</code>	policy	M4
4	Duel and Sandbox repeat scene setup (environment, camera, projectile root, damage numbers)	Two places to fix when the arena changes	Extract a shared <code>Battlefield</code> scene	small	M4
5	Global balance numbers live in code constants (damage table, sailing constants, epic roll scale)	Balancing means editing code, not data	Fine for now; if you start tuning often, move to a single <code>Tuning</code> resource	small	when tuning hurts
6	Pausing is spread across <code>Duel</code> , <code>InventoryScreen</code> and a flag for tests	Easy to get a stuck pause when adding a new overlay	A tiny <code>OverlayStack</code> that counts open overlays and pauses while any is open	small	M4 (more overlays coming)
7	All UI is built in code	Layout tweaks need code edits; hard for an artist	Move to scene files when art arrives; the logic split already supports it	medium	art pass
8	<code>HardpointMount</code> mixes visuals and logic	Real ship art will need to replace the visual half	Separate a <code>MountVisual</code> that real models can supply	small	art pass

#	ISSUE	WHY IT MATTERS	SUGGESTED FIX	EFFORT	WHEN
9	No saving to disk yet	Items and loadouts already serialize; runs do not	Add <code>SaveGame</code> that writes <code>to_dict()</code> with a version number and a backup copy (spec 5.7)	medium	M4
10	Fighters, Q/E/R abilities and EMP are data-only shells	Players can equip items that do nothing	Build the Effect pipeline, then fighters and abilities on top	large	M4
11	Auto weapons (yours and the enemy's) are perfectly accurate	Enemies with auto guns are never clumsy; only manual and lock-on shots have aim error	Optional: give <code>AIProfile</code> an auto-aim error; changes difficulty feel	small	if enemies feel too sharp
12	Tests are slow without a special flag; no automated runner	Easy to forget the flag or skip tests	A one-line script or CI job that runs the standard command	small	any time

Is M4 ready to build?

Yes. Nearly every M4 feature has a landing spot already:

M4 FEATURE	WHERE IT LANDS	ALREADY AVAILABLE
Reward: pick 1 of 3	new <code>LootGenerator</code> (rolls with <code>ItemRoller</code>) + a reward screen that reuses <code>ItemSlot</code> and <code>ItemTooltip</code>	rolling, rarity, tooltips, seeded RNG
Enemies drop what they used	read the defeated ship's <code>loadout.equipped()</code> : they are already real <code>ItemInstances</code>	items rolled at spawn
Refit between duels	the existing refit screen, opened from the result card	built and wired in the duel
Hull damage carries over	store hull points in <code>RunState</code> ; set <code>health.hull</code> after setup	<code>RunState</code> , <code>HealthComponent</code>
Scrap and cargo of 4	a number in <code>RunState</code> ; <code>RunState.cargo_capacity = 4</code> ; a discard-for-scrap action on the existing discard bin	bin, <code>loadout.take()</code>
Working fighters and abilities	the Effect pipeline + <code>AbilityController</code> on the Ship; Q/E/R input actions in <code>PlayerInput</code>	<code>FighterBayData</code> , active names, command API
Saving a run	<code>SaveGame</code> writing <code>RunState</code> + <code>ShipLoadout.to_dict()</code>	dictionaries, unknown-id handling
Three acts, the Port	more <code>ActData</code> files; a Port scene reusing the refit screen	<code>ActData</code> , <code>RunState.advance()</code>

Where the spec stands

A map from the design document's sections to what is built. Status: done partial not yet

SPEC SECTION	STATUS	NOTES
2.1 Sailing (thrust, speed-scaled turning, Hard Burn)	done	plus lateral grip, reverse thrust, burn steering lock (our additions)
2.2 Damage layers and types	done	table in <code>damage_model.gd</code>
2.3 Weapons: arcs, cooldowns, auto / manual / lock-on	done	all four slice turrets
2.4 Actives and fighters (Q / E / R)	not yet	data and UI shells exist; no behaviour (M4)
2.5 Readability: telegraphs, asteroid cover, juice	partial	railgun glow, asteroid cover, hit flash, numbers done; hit-stop, screen shake, sparks, debris not done
3.1 Hulls with typed hardpoints	done	7 hulls
3.2 Items: built-in behaviour + rolled stats, rarity	done	Legendary reserved
3.3 Crew: station bonus + active	partial	bonus real, active is a shell
3.4 Techmods	partial	passives real, EMP active is a shell
3.5 Synergies	not yet	phase P2 in the spec
4.1 Gauntlet: enemy preview, three duels	partial	act + preview + three duels done; Port, hull carry-over not yet
4.2 Enemies: hull + items + AI profile; bosses with phases	done	brawler, sniper, mini-boss with one phase; "drops what they used" is M4
4.3 After each duel: reward, scrap, refit	partial	refit done; reward and scrap are M4
4.4 Meta (materials, blueprints, forge, vault)	not yet	phase P3
5.1-5.3 Architecture, data vs instances, ship scene	done	layout matches the spec
5.4 Effects pipeline	not yet	planned for M4
5.5 Autoloads	partial	ContentDB, RunState (minimal), EventBus done; MetaSave not yet
5.7 Saving	partial	save dictionaries done; file writing not yet
5.8 Seeded randomness	partial	RunState seed and per-duel RNG done; sandbox randomizes by design
6 Vertical slice contents and arena	done	hull, 4 turrets, 2 bays (shells), 2 techmods, 2 crew, 2 AIs, mini-boss, arena with 5 asteroids, sandbox
7 Error handling	partial	content validation, unknown-id tolerance done; loadout validation before each duel and save backups not yet
8 Testing	done	GUT, unit + integration + probe

Glossary

Plain-language definitions, in the order you are likely to need them.

TERM	MEANING HERE
Node	a building block of a Godot scene: a ship, a camera, a label. Nodes form trees.
Scene (.tscn)	a saved tree of nodes you can instance many times: <code>ship.tscn</code> is instanced for every ship.
Resource (.tres)	a saved bundle of data with no behaviour of its own: a hull, a weapon. We call these <i>recipes</i> .
class_name	gives a script a global name so any file can use it (<code>HullData</code> , <code>Ship</code>).
Autoload	a node Godot creates at startup that every script can reach by name: ContentDB, RunState, EventBus.
Signal	a node announcing "something happened" without knowing who listens.
Group	a tag on nodes so you can ask the scene for "all ships".
CharacterBody3D	Godot's body type for things you move yourself and that collide: the Ship.
Marker3D	an invisible position marker in a model; a hardpoint's mount sits on one.
Hardpoint	a typed slot on a hull: turret (small or large, with an arc), fighter bay, techmod or crew seat.
Loadout	what is mounted on each hardpoint, plus the cargo hold (<code>ShipLoadout</code>).
Cargo	spare items in the hold; 12 cells in the sandbox, 4 in a real run.
ItemData / ItemInstance	the recipe for an item / one specific drop with its rarity and rolls.
Roll	a randomly chosen bonus line on a drop, from the item's roll pool.
StatMod	one modifier: stat, flat or percent, amount, and scope.
Scope	who a modifier applies to: this item, all turrets, or the ship.
Potency	the Engineer's bonus that makes techmod effects stronger.
Arc	the angle a turret can fire through, measured from the ship's nose.
Lead	aiming ahead of a moving target so the shot arrives where it will be.
Pursuit point	the spot on a circle around the target that an AI heads for.
Phase	a boss's change of behaviour at a hull threshold.
Act	an ordered list of encounters (the slice's act has three).
Ray-cast	an instant invisible line used to ask "what is along here?"; how projectiles detect hits.
Physics interpolation	Godot smoothing movement between fixed physics steps so motion looks fluid at any frame rate.
GUT	the test framework (Godot Unit Test) vendored in <code>addons/gut</code> .
Fixed-fps run	running the engine in simulated time as fast as possible; used for tests and the probe.
Probe	a script that measures something (duel length) instead of pass/fail testing.
Shell	an item that exists with stats and UI but whose active ability is not implemented yet.
Sandbox	the debug scene for free play and testing; also the current main scene.

Appendix: every script, one line each

Line counts are approximate. Use it like a phone book: find the name, read what it is for.

scripts/data (classes that describe content)

FILE	LINES	PURPOSE
hull_data.gd	61	a hull: handling, health, model, hardpoints
hardpoint_data.gd	23	one slot: type, size, arc, marker name
item_data.gd	30	base class of every item: id, name, mods, roll pool, active
weapon_data.gd	31	a turret weapon: mode, projectile, cooldown, charge, lock, range
techmod_data.gd / crew_data.gd / fighter_bay_data.gd	46	the other item kinds
projectile_data.gd	23	what a weapon fires: speed, damage, type, homing, pierce, blast
stat_mod.gd	23	one modifier with op and scope
stats.gd	25	constants naming every stat
roll_template.gd / roll_pool.gd	16	which bonus lines an item may roll
rarity.gd	28	Common..Legendary: names, colours, roll counts, epic scale
damage.gd / hit.gd / damage_result.gd	49	damage types, a hit in flight, what a hit did per layer
enemy_data.gd / item_spec.gd / phase_data.gd / act_data.gd / ai_profile.gd	77	enemy recipes, boss phases, the act, AI style
duel_result.gd	10	how one duel ended

scripts/systems (pure rules)

FILE	LINES	PURPOSE
damage_model.gd	63	shield, armor, hull maths and the type table
stat_aggregator.gd	32	base + flat, times (1 + percent)
loadout_stats.gd / weapon_stats.gd	67	turn equipped items into modifier lists and weapon numbers
ship_loadout.gd	175	slots + cargo and every move rule; save format
loadout.gd	11	does this item fit this hardpoint
item_instance.gd / item_roller.gd	95	one drop; rolling drops from a seeded generator
arc.gd	33	firing-arc maths
ai_steering.gd	54	pursuit point, steering commands, dodge direction
enemy_factory.gd	48	build a live enemy ship from a recipe
content_validator.gd	164	startup checks on all data
content_loader.gd	26	load every .tres under a folder

scripts/autoload (always on)

FILE	LINES	PURPOSE
content_db.gd	78	find content by id; lists of hulls, weapons, items, enemies
run_state.gd	74	the current run's hull, gear, act and seed
event_bus.gd	9	hit, destroyed, fired, phase started

scripts/ship (what a ship is)

FILE	LINES	PURPOSE
ship.gd	264	the hub: command API, loadout, team, death
sailing_model.gd	114	pure sailing physics
movement_component.gd	58	applies the model to the body
stats_component.gd	42	final stats of a ship
health_component.gd	100	shield, hull, regen, death
hardpoint_mount.gd	128	placeholder visuals of a slot and its barrel
weapon_controller.gd	348	one weapon: all three firing modes
projectile.gd	161	a shot: ray-cast, pierce, home, explode
player_input.gd / mouse_aim.gd	41	keyboard and mouse to commands; cursor to world point
ai_controller.gd	155	the enemy brain
phase_controller.gd	51	boss phase changes
follow_camera.gd	50	top-down follow camera
hit_flash.gd / vfx.gd	60	hit bubble; expanding rings

scripts/arena, duel, debug

FILE	LINES	PURPOSE
arena/arena.gd	49	floor, edge ring, boundary enforcement
duel/duel.gd	187	a duel: preview, fight, result; pause handling
debug/sandbox.gd	350	free-play scene: spawn, swap hulls, time scale
debug/sandbox_panel.gd	138	F2 panel
debug/tuning_panel.gd	142	F1 panel: live sailing sliders
debug/circle_bot.gd	19	debug controller that drives in circles

scripts/ui (screens)

FILE	LINES	PURPOSE
hud.gd	150	speed, health, weapon status, banner, help text
inventory_screen.gd	209	the refit screen
item_slot.gd	129	one drag-and-drop cell

FILE	LINES	PURPOSE
item_tooltip.gd	95	item hover card
stats_panel.gd	81	ship stats with change colours
stat_info.gd / item_visuals.gd	121	names, number formats, colours, badges
duel_ui.gd	241	preview card, enemy bar, banner, result card
damage_numbers.gd	50	floating numbers
health_bar.gd	57	bars above ships